University of Southern Denmark

Department of Mathematics and Computer Science

SPD801: Master Thesis in Computer Science

Formalising πLL in Lean

Author

Mikkel Brix Nielsen
mikke21

Supervisors

Supervisor: Fabrizio Montesi
Cosupervisor: Marco Peressotti
Cosupervisor: Xueying Qin

31st May 2026

Abstract

Words words words words words words words words words

Ord ord ord ord ord ord ord ord ord ord ord

Contents

1 Introduction

The Curry-Howard correspondence establishes a deep connection between logic and computation, where proofs correspond to programs and logical propositions correspond to types. This perspective has been particularly influential in the study of concurrent systems, where linear logic and session types provide a logical foundation for structured communication.

In this setting, linear logic captures resource-sensitive reasoning, while session types describe communication protocols between concurrent processes. The resulting correspondence, often referred to as propositions-as-sessions, provides a principled interpretation of interaction as logical proof transformation, and has been further related to process calculi such as the π-calculus.

This line of work has been further developed by Montesi and Peressotti [2021], who provide a unified account reconciling linear logic, the π-calculus, and their metatheory. Their framework serves as the formal basis for the system πLL considered in this thesis.

Motivation and correctness concerns. Despite the elegance of these theoretical correspondences, their informal and paper-based presentations are prone to subtle errors. In particular, the application of inference rules in linear logic and session-typed systems often relies on implicit ordering conventions and intuitive interpretations of syntax. This can lead to mistakes that are not immediately apparent in written derivations.

As an illustrative example, a small mistake in the application of a typing rule can arise when the intuitive order of names does not match the formal order imposed by the rule. It is tempting to assign types following the order in which names appear syntactically, but a rule may assign types according to the role each name plays instead. Thus, despite the rule itself being correct, its application is easy to get wrong because the intuitive and formal interpretations pull in opposite directions. Such mistakes does not reflect a flaw in the underlying theory, but they are easy to overlook in paper presentations where rule application is guided by intuition rather than by strict syntactic structure. This makes the system a natural candidate for mechanisation, since in proof assistants such as Lean, every rule application must instantiate the required variables and side conditions explicitly.

These observations motivate a more disciplined approach to formal reasoning, where correctness is ensured by construction rather than by inspection.

Why mechanisation. Mechanisation provides a means of bridging this gap. Rather than extending the underlying theory, the goal of mechanisation in this work is to validate and internalise existing results within a proof assistant. This allows us to eliminate ambiguity arising from paper presentation and to ensure that all derivations are checked with respect to a precise formal semantics.

Moreover, mechanised developments have repeatedly demonstrated their ability to uncover small but meaningful inconsistencies in paper proofs, supporting the view that proof assistants serve as a valuable tool for verifying, rather than merely implementing, theoretical frameworks.

Contributions. In this thesis, we present a mechanised formalisation of the π-calculus-based linear logic system πLL as developed by Montesi and Peressotti, with particular emphasis on its multiplicative fragment. The contributions are as follows:

- A formalisation of the syntax and typing system of πLL in Lean, including processes, session types, environments, hyperenvironments, typing judgements, and substitution operations for both names and types.
- A binding architecture based on De Bruijn indices for type variables and a locally nameless representation for process binders, avoiding explicit reasoning up to α-equivalence.
- A formalisation of the operational semantics for the multiplicative fragment of πLL.

- A mechanised development of the session-fidelity proof architecture for the multiplicative fragment, with the remaining proof obligation isolated to the `res` case.

Scope of the formalisation. This thesis focuses on a mechanised reconstruction of the πLL system of Montesi and Peressotti [2021], with particular emphasis on its multiplicative fragment. The syntactic and typing components of πLL have been mechanised for the full system, including processes, session types, typing judgements, and substitution operations. The operational semantics are defined for the multiplicative fragment, for which the session-fidelity development addresses the project objective of relating process evolution to the linear-logic protocols represented by typing derivations. Within this fragment, completing the session-fidelity argument is reduced to the remaining `res` case of $\texttt{typability\_subject\_reduction}_{\texttt{m}}$.

Consequently, metatheoretic results concerning the static typing infrastructure apply to the full system, while the operational and session-fidelity results developed in this thesis are restricted to the multiplicative fragment and remain conditional on completion of the outstanding `res` case. The aim of this work is not to extend the theory, but to provide a mechanised account of an existing theoretical framework, to establish the completed parts of its metatheory in Lean, and to isolate precisely the remaining proof obligation for the targeted session-fidelity result.

2 Background

This section begins by introducing the theoretical foundations underlying the formalisation following the presentation of Montesi and Peressotti [2021], We cover the full πLL calculus, including processes, types, typing, environments / hyperenvironments, and judgements, then restrict attention to the multiplicative fragment πMLL when discussing operational semantics.

Following Montesi and Peressotti [2021], we use *blue* to highlight types and *red* for process terms throughout this thesis.

2.1 Classical Linear Logic, CLL

Linear logic [Girard 1987] is a refinement of classical and intuitionistic logic. In contrast to these systems, where formulas can be freely duplicated or discarded, linear logic restricts structural rules such as weakening and contraction, and instead treats formulas as consumable resources. This yields a resource-sensitive interpretation of logic, where each assumption must be used in a controlled manner [Di Cosmo and Miller 2023]. In this thesis, we work in the classical presentation of linear logic, where sequents are symmetric and duality replaces the distinction between assumptions and conclusions.

In particular, conjunction and disjunction split into multiplicative and additive variants. The multiplicative fragment forms the core of the system and includes the tensor connective $A \otimes B$ and its dual $A \parr B$, together with the corresponding units 1 and $\bot$. These connectives capture composition and interaction of resources.

Intuitively, $A \otimes B$ describes the joint availability of two independent resources, while $A \parr B$ captures their dual form of interaction. Under a process interpretation, $\otimes$ can be read as producing a value of type A and continuing as B, whereas $A \parr B$ corresponds to receiving a value of type A and proceeding as B. The units 1 and $\bot$ represent the absence of further output and input, respectively. A simplified presentation of these rules is given below:

$$\frac{}{\vdash 1}\,1 \qquad \frac{\vdash \Gamma}{\vdash \Gamma, \bot}\,\bot \qquad \frac{\vdash \Gamma, A \quad \vdash \Delta, B}{\vdash \Gamma, \Delta, A \otimes B}\,\otimes \qquad \frac{\vdash \Gamma, A, B}{\vdash \Gamma, A \parr B}\,\parr$$

Here rule $\otimes$ presents a simplified form of the tensor rule from classical linear logic. In general, the tensor connective $A \otimes B$ combines two independent derivations. It requires a proof of A and a proof of B, each obtained from disjoint contexts, and produces a proof of the composite resource $A \otimes B$. Intuitively, this corresponds to combining two independent resources into a single composite resource, and can be read as producing a value of type A and then continuing as B.

The unit rule rule 1 introduces the unit 1, which represents the absence of further obligations. It can be understood as an "empty output", corresponding to a computation that performs no further interaction, and serves as the neutral element of tensor.

Dually, rules $\perp$ and $\parr$ describe the behavior of the connective $A \parr B$ and its unit $\perp$. The $\parr$ rule combines resources within a single context, reflecting a form of interaction where both components are made available to the surrounding context. From an operational perspective, this corresponds to interacting with an input of type A while continuing as B. The unit $\perp$ represents the absence of further input, acting as an "empty input" and serving as the dual neutral element.

For example, two independent instances of the unit 1 can be introduced by the rule 1 and combined via rule $\otimes$ to derive $1 \otimes 1$:

$$\dfrac{\dfrac{}{\vdash 1}\,1 \qquad \dfrac{}{\vdash 1}\,1}{\vdash 1 \otimes 1}\,\otimes$$

Here each leaf introduces one resource independently, and the tensor rule combines them into a single compound resource $1 \otimes 1$. After this step, the individual resources are no longer available separately, reflecting the resource-sensitive nature of the system.

This resource-oriented view of logic provides the foundation for its applications in computer science, particularly in the study of concurrency and session-typed communication. In the following sections, this perspective is used to motivate the process interpretation underlying πLL.

2.2 The π-Calculus: Processes and Communication

The π-calculus [Milner et al. 1992] is a process calculus for modeling concurrent computation with dynamic communication topology, where channel names themselves can be transmitted, allowing the communication network to evolve during execution. We follow the presentation of Montesi and Peressotti [2021], which adopts Wadler's convention of using square brackets '[-]' for output and round parentheses '(-)' for input.

Programs in πMLL are processes (P, Q, R, ...), which communicate over names (x, y, z, ...) representing session endpoints. Sessions are binary (they involve exactly two endpoints), a discipline enforced by the typing system introduced in 2.3. We assume an infinite set of names with decidable equality. The process terms of πMLL are defined by the following grammar:

$$\begin{array}{rcll}
P, Q & ::= & x[y].P & \textit{output } y \textit{ on } x \textit{ and continue as } P \\
& \mid & x(y).P & \textit{input } y \textit{ on } x \textit{ and continue as } P \\
& \mid & x[].P & \textit{output (empty message) on } x \textit{ and continue as } P \\
& \mid & x().P & \textit{input (empty message) on } x \textit{ and continue as } P \\
& \mid & \boldsymbol{\nu} xy\, P & \textit{name restriction (cut)} \\
& \mid & P \mid Q & \textit{parallel composition} \\
& \mid & x[\textsc{l}].P & \textit{select left on } x \textit{ and continue as } P \\
& \mid & x[\textsc{r}].P & \textit{select right on } x \textit{ and continue as } P \\
& \mid & x.\mathsf{case}\{\textsc{l}:P, \textsc{r}:Q\} & \textit{offer a binary choice between } P \textit{ or } Q \textit{ on } x \\
& \mid & x[A].P & \textit{output type } A \textit{ on } x \textit{ and continue as } P \\
& \mid & x(X).P & \textit{input a type as } X \textit{ on } x \textit{ and continue as } P
\end{array}$$

$\mid$	$!x\langle z_1, \ldots, z_n\rangle.\{P\}$	*server (replicable process) on x depending on servers on $z_1 \ldots z_n$*
$\mid$	$x[\textsc{use}].P$	*consume a server on x and continue as P*
$\mid$	$x[\textsc{dup}](x').P$	*duplicate server x as x' in P*
$\mid$	$x[\textsc{disp}].P$	*dispose of a server on x*
$\mid$	$x \leftrightarrow y$	*link x and y*
$\mid$	$\mathbf{0}$	*terminated process*

Term $x[y].P$ allocates a fresh name y, outputs it over x, and proceeds as P. Dually, $x(y).P$ inputs a name over x, binding it to y in P. Terms $x[].P$ and $x().P$ model empty output and input. Restriction $\boldsymbol{\nu}xy\,P$ connects the two endpoints x and y of P to form a session, where the order of x and y is irrelevant and $\boldsymbol{\nu}xy\,P = \boldsymbol{\nu}yx\,P$. Parallel composition $P \mid Q$ runs P and Q concurrently, and $\mathbf{0}$ is the terminated process, acting as the unit of parallel composition.

Example 2.1. Consider the process $P \triangleq \boldsymbol{\nu}xz\,(x[].\mathbf{0} \mid z().y[].\mathbf{0})$. This process creates a private session between the restricted endpoints x and z. The left process, $x[].\mathbf{0}$, signals termination on x before terminating, while the right process, $z().y[].\mathbf{0}$, waits for a termination signal on z before proceeding by sending a termination signal on y and then terminating.

This calculus provides the operational foundation for πLL. In the following section, we introduce the type system that governs how names are used, ensuring that each session endpoint is used exactly according to its protocol.

2.3 Types and Propositions-as-Sessions Correspondence

In πLL, session types ($A, B, C, \ldots$) are linear logic propositions [Caires and Pfenning 2010; Montesi and Peressotti 2021; Wadler 2014]. Each type describes the communication protocol of a channel endpoint, and duality, the key structural feature of classical linear logic, ensures that the two endpoints of every session implement complementary protocols.

The correspondence between linear logic and session types used in this thesis follows the presentation of Montesi and Peressotti [2021], which builds on the accounts of Carbone et al. [2016] and Wadler [2014], is captured directly by the type grammar. Each connective denotes a communication action, where $A \otimes B$ describes sending a value of type A and continuing as B, while its dual $A \parr B$ describes receiving. The units 1 and $\bot$, again, represent the absence of out and input, respectively. The additive connectives $A \oplus B$ and $A \,\&\, B$ capture choice in the sense of selection and offering of alternatives, while the exponentials $!A$ and $?A$ govern replication. The $\forall X.A$ and $\exists X.A$ allow polymorphic session behavior. The type grammar of πLL is defined as follows:

$A, B ::=$	$A \otimes B$	*send A, proceed as B*	$\mid$	$A \parr B$	*receive A, proceed as B*
$\mid$	1	*empty output, unit for $\otimes$*	$\mid$	$\bot$	*empty input, unit for $\parr$*
$\mid$	$A \,\&\, B$	*offer A or B*	$\mid$	$A \oplus B$	*select A or B*
$\mid$	$\exists X.A$	*existential, type output*	$\mid$	$\forall X.A$	*universal, type input*
$\mid$	$?A$	*client request*	$\mid$	$!A$	*server accept*
$\mid$	X	*type variable*	$\mid$	${}^{\perp}X$	*dual of type variable*

Duality is defined structurally on session types by exchanging each connective with its dual and forms an involution, i.e. $(A^{\perp})^{\perp} = A$. Consequently, the endpoints of a well-typed session are always assigned dual types, ensuring they are compatibility:

$$(A \otimes B)^{\perp} = A^{\perp} \parr B^{\perp} \qquad 1^{\perp} = \bot \qquad (A \oplus B)^{\perp} = A^{\perp} \,\&\, B^{\perp}$$

$$(!A)^\perp = ?A^\perp \qquad (\exists X.A)^\perp = \forall X.A^\perp \qquad (X)^\perp = X^\perp$$

2.4 Environments, Hyperenvironments and Judgements

Typing in πLL is organised into two layers. Environments track how individual names are typed, while hyperenvironments track which names are used independently in parallel.

Following the presentation in Montesi and Peressotti [2021], *environments* $(\Gamma, \Delta, \Xi, \dots)$ are defined as lists allowing exchange, meaning order is irrelevant. They can be written as $\Gamma = x_1 : A, \dots, x_n : A_n$, where each x_i is associated to its respective A_i, for $1 \le i \le n$. Environments can be merged as long as they do not share any names for example assume Γ and Δ do not share any names, then Γ, Δ is the environment containing exactly the associations in Γ and Δ regardless of ordering. Environments act as a partial commutative monoid, using "," as the sum and the empty environment $\bullet$ as unit:

$$\Gamma, \bullet = \Gamma \qquad \Gamma, \Delta = \Delta, \Gamma \qquad (\Gamma, \Delta), \Xi = \Gamma, (\Delta, \Xi)$$

Environments dictate how names are used, but it does not distinguish whether names are used independently or in parallel. That role is handled by handled by *hyperenvironments* $(\mathcal{G}, \mathcal{H}, \mathcal{I}, \dots)$, which are collections of environments joined using the parallel operator '$\mid$'. They can be written as follows $\mathcal{G} = \Gamma_1, \dots, \Gamma_n$, where $\Gamma_1, \dots, \Gamma_n$ is a collection of environments. Given $\mathcal{G}$ and $\mathcal{H}$ having no names in common then $\mathcal{G} \mid \mathcal{H}$ is the hyperenvironment containing exactly the environments of $\mathcal{G}$ and $\mathcal{H}$, ignoring order. Like environments, all names in a hyperenvironment are unique, they can be combined as long as they do not share any names, and they act as a partial commutative monoid using '$\mid$' the sum and $\varnothing$ as unit, where $\varnothing$ is the hyperenvironment containing no environments:

$$\mathcal{G} \mid \varnothing = \mathcal{G} \qquad \mathcal{G} \mid \mathcal{H} = \mathcal{H} \mid \mathcal{G} \qquad (\mathcal{G} \mid \mathcal{H}) \mid \mathcal{I} = \mathcal{G} \mid (\mathcal{H} \mid \mathcal{I})$$

Judgements, written as $\vdash P :: \mathcal{G}$, states that the process P uses its free names in accordance with $\mathcal{G}$ and can be derived using the inference rules displayed in Figure 1.

Typing Derivations $(\mathcal{D}, \mathcal{E}, \mathcal{F}, \dots)$ are written as $\vdash \overset{\mathcal{D}}{P} :: \mathcal{G}$ and are read as the derivation $\mathcal{D}$ having the judgement $\vdash P :: \mathcal{G}$ as its conclusion. The meaning of this statement is that the process, P, appearing in the judgement concluding a derivation is well-typed.

The multiplicative rules form the core of the system. Rule CUT composes two processes in P sharing dual endpoints $x : A$ and $y : A^\perp$, hiding them via restriction. This is the logical equivalent of communication. Rule MIX and rule MIX_0 compose independent processes in parallel and introduce the terminated process, respectively.

Rule $\otimes$ and rule $⅋$ type output and input. Note that rule $\otimes$ types the sent name y in a *separate* environment from the continuation, reflecting that the two are independent resources. By contrast, rule $⅋$ keeps y and the continuation x in the *same* environment, reflecting their sequential dependency. The units rule 1 and rule $\perp$ type empty output and empty input, where rule 1 requires the continuation to be well-typed in the empty hyperenvironment, meaning P is necessarily semantically equivalent to $\mathbf{0}$.

The additive rules (rule $\oplus_1$, rule $\oplus_2$, and rule $\&$) type selection and offering of alternative processing paths. Note that rule $\&$ requires both branches to be typed in the *same* context Γ, reflecting that the choice of branch is made by the other endpoint.

The exponential rules type server replication. The rule ! requires all free names except x to be typed using ?, enforcing that a server only depends on other servers. The rule ?, rule W, and rule C allow a client to use, discard, or duplicate a server respectively.

The rule AX types a link between two dual endpoints, $x \leftrightarrow y$, forwarding all messages sent on x safely to y and vice versa. Rule $\exists$ and rule $\forall$ type polymorphic communication, where the side

Multiplicative Fragment

$$\dfrac{\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^{\perp}}{\vdash \nu xy P :: \mathcal{G} \mid \Gamma, \Delta}\ \text{CUT} \qquad \dfrac{\vdash P :: \mathcal{G} \qquad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\ \text{MIX} \qquad \dfrac{}{\vdash \mathbf{0} :: \varnothing}\ \text{MIX}_0$$

$$\dfrac{\vdash P :: \Gamma, y : A \mid \Delta, x : B}{\vdash x[y].P :: \Gamma, \Delta, x : A \otimes B}\ \otimes \qquad \dfrac{\vdash P :: \varnothing}{\vdash x[].P :: x : 1}\ 1 \qquad \dfrac{\vdash P :: \Gamma, y : A, x : B}{\vdash x(y).P :: \Gamma, x : A \parr B}\ \parr \qquad \dfrac{\vdash P :: \Gamma}{\vdash x().P :: \Gamma, x : \bot}\ \bot$$

Additional rules for Additives, Quantifiers and Exponentials

$$\dfrac{\vdash P :: \Gamma, x : A}{\vdash x[\text{L}].P :: \Gamma, x : A \oplus B}\ \oplus_1 \qquad \dfrac{\vdash P :: \Gamma, x : B}{\vdash x[\text{R}].P :: \Gamma, x : A \oplus B}\ \oplus_2 \qquad \dfrac{\vdash P :: \Gamma, x : A \qquad \vdash Q :: \Gamma, x : B}{\vdash x.\mathsf{case}\{\text{L}{:}P, \text{R}{:}Q\} :: \Gamma, x : A \,\&\, B}\ \&$$

$$\dfrac{}{\vdash x \leftrightarrow y :: x : A^{\perp}, y : A}\ \text{AX} \qquad \dfrac{\vdash P :: \Gamma, x : A}{\vdash x[\text{USE}].P :: \Gamma, x : ?A}\ ? \qquad \dfrac{\vdash P :: \Gamma}{\vdash x[\text{DISP}].P :: \Gamma, x : ?A}\ \text{W} \qquad \dfrac{\vdash P :: \Gamma, x : ?A, x' : ?A}{\vdash x[\text{DUP}](x').P :: \Gamma, x : ?A}\ \text{C}$$

$$\dfrac{\vdash P :: z_1 : ?B_1, \dots, z_n : ?B_n, x : A}{\vdash !x\langle z_1, \dots, z_n\rangle.\{P\} :: z_1 : ?B_1, \dots, z_n : ?B_n, x : !A}\ ! \qquad \dfrac{\vdash P :: \Gamma, x : B\{A/X\}}{\vdash x[A].P :: \Gamma, x : \exists X.B}\ \exists \qquad \dfrac{\vdash P :: \Gamma, x : B \qquad X \notin \mathrm{ft}(\Gamma)}{\vdash x(X).P :: \Gamma, x : \forall X.B}\ \forall$$

Fig. 1. πLL, typing rules.

condition $X \notin \mathrm{ft}(\Gamma)$ in rule $\forall$, ensures the introduced type variable, X, does not accendentally shadow one already existing in Γ.

Example 2.2. The process from Example 2.1 has the typing $\vdash \nu zx\,(z().y[].\mathbf{0} \mid x[].\mathbf{0}) :: y : 1$, as shown by the derivation below:

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\ \text{MIX}_0}{\vdash x[].\mathbf{0} :: x : 1}\ 1 \qquad \dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\ \text{MIX}_0}{\vdash y[].\mathbf{0} :: y : 1}\ 1}{\vdash z().y[].\mathbf{0} :: y : 1, z : \bot}\ \bot}{\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x : 1 \mid y : 1, z : \bot}\ \text{MIX}}{\vdash \nu xz\,(x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: y : 1}\ \text{CUT}$$

Note that, for the typing envrionment $x : 1 \mid y : 1, z : \bot$ to align with the premise of rule CUT, we utilise the monoidal properties of environments and hyperenvironments. In particular, using their identity laws, we instantiate the meta-variables of the rule as $\mathcal{G} = \varnothing$ and $\Gamma = \bullet$. Under these instantiations, the premise of rule CUT: $\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^{\perp}$ can be viewed as $x : A \mid \Delta, y : A^{\perp}$. This effectively instantiates the CUT over the dual types $A = 1$ and $A^{\perp} = \bot$ on channels x and y, with $\Gamma = y : 1$, while ensuring the restricted endpoints implement dual session behaviours, fulfilling the requirements for creating a valid CUT (session).

2.5 Operational Semantics

The operational semantics of πLL are given as a labelled transition system (LTS) defined over *typing derivations*, in which processes evolve by performing actions on their free names. The semantics follow a Structural Operational Semantics (SOS) style presentation and form the basis for the metatheoretic results of the calculus. We will be focusing on the multiplicative fragment for this part, which is given in Figure 2. The full LTS presented by Montesi and Peressotti [2021].

The operational semantics rely explicitly on the structural equivalence (rule $=_\alpha$) to rename restricted variables via α-equivalence prior to a transition. In paper-based presentations, α-equivalence

$$\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \varnothing\end{array}}{\vdash x[].P :: x:1}\,1 \xrightarrow{x[]} \begin{array}{c}\mathcal{D}\\ \vdash P :: \varnothing\end{array} \qquad \dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \Gamma, x':A \mid \Delta, x:B\end{array}}{\vdash x[x'].P :: \Gamma, \Delta, x:A\otimes B}\,\otimes \xrightarrow{x[x']} \begin{array}{c}\mathcal{D}\\ \vdash P :: \Gamma, x':A \mid \Delta, x:B\end{array}$$

$$\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \Gamma\end{array}}{\vdash x().P :: \Gamma, x:\bot}\,\bot \xrightarrow{x()} \begin{array}{c}\mathcal{D}\\ \vdash P :: \Gamma\end{array} \qquad \dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \Gamma, x':A, x:B\end{array}}{\vdash x(x').P :: \Gamma, x:A\parr B}\,\parr \xrightarrow{x(x')} \begin{array}{c}\mathcal{D}\\ \vdash P :: \Gamma, x':A, x:B\end{array}$$

$$\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\end{array} \xrightarrow{l} \begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}'\end{array} \qquad \mathrm{i}(l)\cap \mathrm{f}(Q)=\emptyset}{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\end{array} \quad \begin{array}{c}\mathcal{E}\\ \vdash Q :: \mathcal{H}\end{array}}{\vdash P \mid Q :: \mathcal{G}\mid\mathcal{H}}\,\textsc{mix} \xrightarrow{l} \dfrac{\begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}'\end{array} \quad \begin{array}{c}\mathcal{E}\\ \vdash Q :: \mathcal{H}\end{array}}{\vdash P' \mid Q :: \mathcal{G}'\mid\mathcal{H}}\,\textsc{mix}}\,\textsc{par}_1$$

$$\dfrac{\begin{array}{c}\mathcal{E}\\ \vdash Q :: \mathcal{H}\end{array} \xrightarrow{l} \begin{array}{c}\mathcal{E}'\\ \vdash Q' :: \mathcal{H}'\end{array} \qquad \mathrm{i}(l)\cap \mathrm{f}(P)=\emptyset}{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\end{array} \quad \begin{array}{c}\mathcal{E}\\ \vdash Q :: \mathcal{H}\end{array}}{\vdash P \mid Q :: \mathcal{G}\mid\mathcal{H}}\,\textsc{mix} \xrightarrow{l} \dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\end{array} \quad \begin{array}{c}\mathcal{E}'\\ \vdash Q' :: \mathcal{H}'\end{array}}{\vdash P \mid Q' :: \mathcal{G}\mid\mathcal{H}'}\,\textsc{mix}}\,\textsc{par}_2$$

$$\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\end{array} \xrightarrow{l} \begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}'\end{array} \quad \begin{array}{c}\mathcal{E}\\ \vdash Q :: \mathcal{H}\end{array} \xrightarrow{l'} \begin{array}{c}\mathcal{E}'\\ \vdash Q' :: \mathcal{H}'\end{array} \qquad \mathrm{i}(l|l')\cap \mathrm{f}(P\mid Q)=\emptyset}{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\end{array} \quad \begin{array}{c}\mathcal{E}\\ \vdash Q :: \mathcal{H}\end{array}}{\vdash P \mid Q :: \mathcal{G}\mid\mathcal{H}}\,\textsc{mix} \xrightarrow{l|l'} \dfrac{\begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}'\end{array} \quad \begin{array}{c}\mathcal{E}'\\ \vdash Q' :: \mathcal{H}'\end{array}}{\vdash P' \mid Q' :: \mathcal{G}'\mid\mathcal{H}'}\,\textsc{mix}}\,\textsc{syn}$$

$$\dfrac{P =_\alpha Q \quad \begin{array}{c}\mathcal{E}\\ \vdash Q :: \mathcal{G}\end{array} \xrightarrow{l} \begin{array}{c}\mathcal{E}'\\ \vdash Q' :: \mathcal{G}'\end{array}}{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\end{array} \xrightarrow{l} \begin{array}{c}\mathcal{E}'\\ \vdash Q' :: \mathcal{G}'\end{array}}\,=_\alpha \qquad \dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\mid x:1\mid \Gamma, y:\bot\end{array} \xrightarrow{x[]|y()} \begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}\mid\Gamma\end{array}}{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\mid x:1\mid \Gamma, y:\bot\end{array}}{\vdash \nu xy\, P :: \mathcal{G}\mid\Gamma}\,\textsc{cut} \xrightarrow{\tau} \begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}\mid\Gamma\end{array}}\,1\bot$$

$$\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\mid \Gamma,\Delta, x:A\otimes B\mid \Xi, y:A^\perp\parr B^\perp\end{array} \xrightarrow{x[x']|y(y')} \begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}\mid \Gamma, x:B\mid \Delta, x':A\mid \Xi, y:B^\perp, y':A^\perp\end{array}}{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\mid \Gamma,\Delta, x:A\otimes B\mid \Xi, y:A^\perp\parr B^\perp\end{array}}{\vdash \nu xy\, P :: \mathcal{G}\mid \Gamma,\Delta,\Xi}\,\textsc{cut} \xrightarrow{\tau} \dfrac{\dfrac{\begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}\mid \Gamma, x:B\mid \Delta, x':A\mid \Xi, y:B^\perp, y':A^\perp\end{array}}{\vdash \nu x'y'\, P' :: \mathcal{G}\mid \Gamma, x:B\mid \Delta,\Xi, y:B^\perp}\,\textsc{cut}}{\vdash \nu xy\,\nu x'y'\, P' :: \mathcal{G}\mid \Gamma,\Delta,\Xi}\,\textsc{cut}}\,\otimes\parr$$

$$\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\mid \Gamma, x:A\mid \Delta, y:A^\perp\end{array} \xrightarrow{l} \begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}'\mid \Gamma', x:A\mid \Delta', y:A^\perp\end{array} \qquad x,y\notin \mathrm{f}(l)\cup\mathrm{i}(l)}{\dfrac{\begin{array}{c}\mathcal{D}\\ \vdash P :: \mathcal{G}\mid \Gamma, x:A\mid \Delta, y:A^\perp\end{array}}{\vdash \nu xy\, P :: \mathcal{G}\mid\Gamma,\Delta}\,\textsc{cut} \xrightarrow{l} \dfrac{\begin{array}{c}\mathcal{D}'\\ \vdash P' :: \mathcal{G}'\mid \Gamma', x:A\mid \Delta', y:A^\perp\end{array}}{\vdash \nu xy\, P' :: \mathcal{G}'\mid\Gamma',\Delta'}\,\textsc{cut}}\,\textsc{res}$$

Fig. 2. πMLL, transition rules for derivations.

is often handled implicitly through Barendregt's Variable Convention. In a mechanised setting, however, reasoning over named variables requires freshness conditions, capture-avoiding substitution, and explicit renaming lemmas, introducing substantial proof overhead [Aydemir et al. 2005; Charguéraud 2012]. As discussed in Section 3, this motivates the use of a different binding representation for the syntax of πLL, one designed to reduce explicit α-equivalence reasoning and simplify the treatment of bound variables.

Actions are defined in the set $Act = \{x[], x(), x[y], x(y) \mid x, y \text{ names}\}$, representing empty output, empty input, name output, and name input respectively. These come together with the silent label τ for internal communication and the parallel composition of labels $l \mid l'$ to form the set of transition labels, defined as $Lbl = \{\tau, l, (l \| l') \mid l, l' \in Act, \mathrm{i}(l) \cap \mathrm{i}(l') = \emptyset\}$. The restriction imposed on the parallel composition of labels exists to ensure two labels do not introduce the same name, which would break linear resource management.

The free and introduced names of labels are defined by $\mathrm{f}(x[]) = \mathrm{f}(x()) = \mathrm{f}(x[y]) = \mathrm{f}(x(y)) = \{x\}$, $\mathrm{f}(\tau) = \mathrm{i}(\tau) = \emptyset$, and $\mathrm{f}(l \mid l') = \mathrm{f}(l) \cup \mathrm{f}(l')$, and introduced names by $\mathrm{i}(x[]) = \mathrm{i}(x()) = \emptyset$, $\mathrm{i}(x[y]) = \mathrm{i}(x(y)) = \{y\}$, and $\mathrm{i}(l \mid l') = \mathrm{i}(l) \cup \mathrm{i}(l')$. The parallel label $l \mid l'$ additionally requires $\mathrm{i}(l) \cap \mathrm{i}(l') = \emptyset$, ensuring no two components introduce the same name.

Example 2.3. Figure 3 shows a possible execution for the derivation of the process from Example 2.1.

Each logical rule (1, $\perp$, $\otimes$, $⅋$) has a corresponding transition axiom. These follow the prefix-continuation pattern $\pi.P \xrightarrow{\pi} P$, where performing the action π leads to the remainder of the process. Rule SYN allows for the pairing of concurrent actions via $l \| l'$, which is used to facilitate communication by letting two endpoints connected by a restriction perform compatible actions. When two compatible actions are performed over endpoints connected by a restriction, rules $1\perp$ and $\otimes⅋$ simplify the term into an internal τ-transition, modeling the synchronisation of the session.

2.6 Process Semantics and Erasure

While the typing derivations of πLL dictate the valid interactions within a session, the underlying process terms possess a standard, untyped operational behavior that closely mirrors the classic π-calculus. To formally relate the typed derivations to their untyped execution, we follow Montesi and Peressotti [2021] and define an erasure function, $proc : Der \to Proc$, which strips away the typing information from a derivation. Recursively, $proc$ maps the head of a typing derivation to the specific process constructor it introduces, continuing structurally onto the process's continuation. For any well-typed πLL term, $proc(\mathcal{D})$ effectively extracts the pure process term from the derivation's conclusion.

Example 2.4. Consider the well-typed process P formed by adding a third parallel component to the derivation of Example 2.2 before the cut (A derivation of which can be seen on Figure 4):

$$P = \boldsymbol{\nu} xz\,(w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0})$$

The component containing w and v is independent from the component containing x, z, and y. This allows their respective actions to interleave freely, resulting in the following execution traces, illustrating the concurrency of the calculus:

$$P \xrightarrow{\tau}\xrightarrow{w()}\xrightarrow{v[]}\xrightarrow{y[]} \mathbf{0} \qquad P \xrightarrow{\tau}\xrightarrow{w()}\xrightarrow{y[]}\xrightarrow{v[]} \mathbf{0} \qquad P \xrightarrow{\tau}\xrightarrow{y[]}\xrightarrow{w()}\xrightarrow{v[]} \mathbf{0}$$

$$P \xrightarrow{w()}\xrightarrow{v[]}\xrightarrow{\tau}\xrightarrow{y[]} \mathbf{0} \qquad P \xrightarrow{w()}\xrightarrow{\tau}\xrightarrow{v[]}\xrightarrow{y[]} \mathbf{0} \qquad P \xrightarrow{w()}\xrightarrow{\tau}\xrightarrow{y[]}\xrightarrow{v[]} \mathbf{0}$$

$$
\begin{prooftree}
\AxiomC{}
\RightLabel{MIX$_0$}
\UnaryInfC{$\vdash \mathbf{0} :: \varnothing$}
\RightLabel{1}
\UnaryInfC{$\vdash x[].\mathbf{0} :: x : 1$}
\AxiomC{}
\RightLabel{MIX$_0$}
\UnaryInfC{$\vdash \mathbf{0} :: \varnothing$}
\RightLabel{1}
\UnaryInfC{$\vdash y[].\mathbf{0} :: y : 1$}
\RightLabel{$\bot$}
\UnaryInfC{$\vdash z().y[].\mathbf{0} :: y : 1, z : \bot$}
\RightLabel{MIX}
\BinaryInfC{$\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x : 1 \mid y : 1, z : \bot$}
\RightLabel{CUT}
\UnaryInfC{$\vdash \nu xz\,(x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: y : 1$}
\end{prooftree}
$$

$\downarrow \tau$ (by rules SYN and $1\bot$, and the transition axioms for rules 1 and $\bot$)

$$
\begin{prooftree}
\AxiomC{}
\RightLabel{MIX$_0$}
\UnaryInfC{$\vdash \mathbf{0} :: \varnothing$}
\AxiomC{}
\RightLabel{MIX$_0$}
\UnaryInfC{$\vdash \mathbf{0} :: \varnothing$}
\RightLabel{1}
\UnaryInfC{$\vdash y[].\mathbf{0} :: y : 1$}
\RightLabel{MIX}
\BinaryInfC{$\vdash \mathbf{0} \mid y[].\mathbf{0} :: \varnothing \mid y : 1$}
\end{prooftree}
$$

$\downarrow y[]$ (by rule PAR$_2$, the transition axiom for rule 1)

$$
\begin{prooftree}
\AxiomC{}
\RightLabel{MIX$_0$}
\UnaryInfC{$\vdash \mathbf{0} :: \varnothing$}
\AxiomC{}
\RightLabel{MIX$_0$}
\UnaryInfC{$\vdash \mathbf{0} :: \varnothing$}
\RightLabel{MIX}
\BinaryInfC{$\vdash \mathbf{0} \mid \mathbf{0} :: \varnothing \mid \varnothing$}
\end{prooftree}
$$

$\downarrow \equiv$ (by process congruence, and the monoidal structure of typing environments)

$$
\begin{prooftree}
\AxiomC{}
\RightLabel{MIX$_0$}
\UnaryInfC{$\vdash \mathbf{0} :: \varnothing$}
\end{prooftree}
$$

Fig. 3. An execution for the derivation in Example 2.2.

$$
\begin{prooftree}
\AxiomC{}
\RightLabel{MIX$_0$}
\UnaryInfC{$\vdash \mathbf{0} :: \varnothing$}
\RightLabel{1}
\UnaryInfC{$\vdash v[].\mathbf{0} :: v : 1$}
\RightLabel{$\bot$}
\UnaryInfC{$\vdash w().v[].\mathbf{0} :: v : 1, w : \bot$}
\AxiomC{}
\RightLabel{MIX$_0$}
\UnaryInfC{$\vdash \mathbf{0} :: \varnothing$}
\RightLabel{1}
\UnaryInfC{$\vdash x[].\mathbf{0} :: x : 1$}
\AxiomC{}
\RightLabel{MIX$_0$}
\UnaryInfC{$\vdash \mathbf{0} :: \varnothing$}
\RightLabel{1}
\UnaryInfC{$\vdash y[].\mathbf{0} :: y : 1$}
\RightLabel{$\bot$}
\UnaryInfC{$\vdash z().y[].\mathbf{0} :: y : 1, z : \bot$}
\RightLabel{MIX}
\BinaryInfC{$\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x : 1 \mid y : 1, z : \bot$}
\RightLabel{MIX}
\BinaryInfC{$\vdash (w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: v : 1, w : \bot \mid x : 1 \mid y : 1, z : \bot$}
\RightLabel{CUT}
\UnaryInfC{$\vdash \nu xz\,(w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: v : 1, w : \bot \mid y : 1$}
\end{prooftree}
$$

Fig. 4. A derivation for the process $\nu xz\, w().y[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0}$

The semantics for process terms must inherently agree with the semantics of derivations for well-typed terms. As formulated by Montesi and Peressotti [2021], this relationship is established functorially. Let $\mathcal{P}(X) = \{X' \mid X' \subseteq X\}$ and $\mathcal{L}(X) = X \times Lbl$ be endofunctors representing the states and labeled transitions, respectively. The erasure function *proc* acts as a homomorphism from LTS of derivations (lts_d) to the LTS of processes (lts_p), ensuring that the square of these LTS mappings commutes.

In the context of operational semantics, this functorial homomorphism yields an Erasure theorem. It guarantees that the Labeled Transition System for derivations and the SOS for untyped processes simulate each other exactly:

Theorem 2.5 (Erasure). *The process LTS (lts_p) enjoys erasure with respect to the derivation LTS (lts_d). Concretely, for any valid typing derivation $\mathcal{D}$ and label $l \in Lbl$:*

(1) ***Soundness of Erasure:*** *If $\mathcal{D} \xrightarrow{l} \mathcal{D}'$, then $proc(\mathcal{D}) \xrightarrow{l} proc(\mathcal{D}')$.*

$$x[x'].P \xrightarrow{x[x']} P \qquad x(x').P \xrightarrow{x(x')} P \qquad x[].P \xrightarrow{x[]} P \qquad x().P \xrightarrow{x()} P$$

$$\frac{P \xrightarrow{l} P' \quad \mathrm{i}(l) \cap \mathrm{f}(Q) = \emptyset}{P \mid Q \xrightarrow{l} P' \mid Q}\ \mathrm{PAR}_1 \qquad \frac{Q \xrightarrow{l} Q' \quad \mathrm{i}(l) \cap \mathrm{f}(P) = \emptyset}{P \mid Q \xrightarrow{l} P \mid Q'}\ \mathrm{PAR}_2$$

$$\frac{P \xrightarrow{l} P' \quad Q \xrightarrow{l'} Q' \quad \mathrm{i}(l|l') \cap \mathrm{f}(P \mid Q) = \emptyset}{P \mid Q \xrightarrow{l|l'} P' \mid Q'}\ \mathrm{SYN} \qquad \frac{P \xrightarrow{x[x']|y(y')} P'}{\nu xy\, P \xrightarrow{\tau} \nu xy\, \nu x'y'\, P'}\ \otimes⅋$$

$$\frac{P \xrightarrow{x[]|y()} P'}{\nu xy\, P \xrightarrow{\tau} P'}\ 1\bot \qquad \frac{P \xrightarrow{l} P' \quad x, y \notin \mathrm{f}(l) \cup \mathrm{i}(l)}{\nu xy\, P \xrightarrow{l} \nu xy\, P'}\ \mathrm{RES} \qquad \frac{P =_\alpha Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'}\ =_\alpha$$

$$\begin{aligned} \mathrm{i}(l|l') &= \mathrm{i}(l) \cup \mathrm{i}(l') \\ \mathrm{f}(l|l') &= \mathrm{f}(l) \cup \mathrm{f}(l') \\ \mathrm{i}(\tau) &= \emptyset \\ \mathrm{f}(\tau) &= \emptyset \\ \mathrm{i}(x[]) = \mathrm{i}(x()) &= \emptyset \\ \mathrm{f}(x[]) = \mathrm{f}(x()) &= \{x\} \\ \mathrm{i}(x[x']) = \mathrm{i}(x(x')) &= \{x'\} \\ \mathrm{f}(x[x']) = \mathrm{f}(x(x')) &= \{x\} \end{aligned}$$

Fig. 5. πMLL, transition rules for processes.

(2) ***Completeness of Erasure:*** *If proc*$(\mathcal{D}) \xrightarrow{l} P'$*, then there exists a derivation* $\mathcal{D}'$ *such that* $\mathcal{D} \xrightarrow{l} \mathcal{D}'$ *and proc*$(\mathcal{D}') = P'$.

Corollary 2.6 (Typability Preservation). *If P is well-typed and $P \xrightarrow{l} P'$, then P' is well-typed.*

This theorem allows us to define the operational target for πLL processes without the syntactic overhead of typing environments. Using the erasure formulation, we can transform the operational semantics for derivations directly into an SOS specification for processes.

The process LTS must state freshness and name-disjointness conditions explicitly because it describes process evolution independently of typing information. For example, synchronising actions must not introduce the same fresh name, as expressed by conditions such as $i(l) \cap i(l') = \emptyset$. For processes arising from valid typing derivations, these obligations are supported by the linear resource structure of πLL: typing restricts the combination of hyperenvironment components to disjoint domains and accounts for the names used by the process. Thus the process semantics records the operational side conditions explicitly, while the typed semantics explains why they are satisfied by well-typed execution. The corresponding evolution of these typing resources is described by the environment semantics introduced next.

2.7 Semantics of Typing Environments

Just as processes evolve through computation, the typing environments that govern them must also evolve to accurately track the state of a session. In πLL, types capture communication protocols. Consequently, the transition rules for typing environments specify exactly which actions are permitted and how the available resources are consumed by those actions.

Similar to how we defined the process erasure function, we again follow Montesi and Peressotti [2021] and define a function $env : Der \longrightarrow Env$ that maps a valid typing derivation directly to the hyperenvironment in its conclusion (i.e., $env(\vdash P :: \mathcal{G}) = \mathcal{G}$). Using this extraction, Montesi and Peressotti [2021] derive a LTS specifically for typing environments (lts_e), shown in Figure 6.

In this LTS, observable actions actively consume resources from the environment. For example, if a hyperenvironment contains a waiting channel $x\colon \bot$, performing an input action $x()$ will consume this type, removing it from the resulting environment. Conversely, internal communications do not alter the available external resources. This is formalised by the τ-reflexivity axiom, $\mathcal{G} \mid \Gamma \xrightarrow{\tau} \mathcal{G} \mid \Gamma$, which states that any valid hyperenvironment transitions to itself under a silent τ-action.

The rules in Figure 6 define which transitions are available at the environment level. For a well-typed process, however, process behaviour and resource evolution must step in unison such

$$x : 1 \xrightarrow{x[]} \varnothing \qquad \Gamma, x : \bot \xrightarrow{x()} \Gamma \qquad \Gamma, \Delta, x : A \otimes B \xrightarrow{x[x']} \Gamma, x' : A \mid \Delta, x : B \qquad \Gamma, x : A \parr B \xrightarrow{x(x')} \Gamma, x' : A, x : B$$

$$\frac{\mathcal{G} \xrightarrow{l} \mathcal{G}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l} \mathcal{G}' \mid \mathcal{H}}\ \text{PAR}_1 \qquad \frac{\mathcal{H} \xrightarrow{l} \mathcal{H}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l} \mathcal{G} \mid \mathcal{H}'}\ \text{PAR}_2 \qquad \frac{\mathcal{G} \xrightarrow{l} \mathcal{G}' \quad \mathcal{H} \xrightarrow{l'} \mathcal{H}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l \| l'} \mathcal{G}' \mid \mathcal{H}'}\ \text{SYN}$$

$$\frac{\mathcal{G} \mid \Gamma, x : A^\perp \mid \Delta, y : A \xrightarrow{l} \mathcal{G}' \mid \Gamma', x : A^\perp \mid \Delta', y : A}{\mathcal{G} \mid \Gamma, \Delta \xrightarrow{l} \mathcal{G}' \mid \Gamma', \Delta'}\ \text{RES} \qquad \frac{\mathcal{G} \mid x : 1 \mid \Gamma, y : \bot \xrightarrow{x[] \| y()} \mathcal{G} \mid \Gamma}{\mathcal{G} \mid \Gamma \xrightarrow{\tau} \mathcal{G} \mid \Gamma}\ 1\bot$$

$$\frac{\mathcal{G} \mid \Gamma, \Delta, x : A \otimes B \mid \Xi, y : A^\perp \parr B^\perp \xrightarrow{x[x'] \| y(y')} \mathcal{G} \mid \Gamma, x : B \mid \Delta, x' : A \mid \Xi, y : B^\perp, y' : A^\perp}{\mathcal{G} \mid \Gamma, \Delta, \Xi \xrightarrow{\tau} \mathcal{G} \mid \Gamma, \Delta, \Xi}\ \otimes\parr$$

Fig. 6. πMLL, transition rules for typing environments.

that whenever the process performs an action, its typing environment must be able to perform the corresponding transition. This correspondence is the property of **session fidelity**. Just as the *proc* projection extracts the process behaviour of a typed transition, the *env* projection extracts its corresponding environment evolution.

Theorem 2.7 (Session Fidelity). *Let $\mathcal{D}$ be a valid typing derivation such that $env(\mathcal{D}) = \mathcal{G}$ and $proc(\mathcal{D}) = P$. If the process transitions such that $P \xrightarrow{l} P'$, then the environment can mimic this transition $\mathcal{G} \xrightarrow{l} \mathcal{G}'$, and there exists a new valid derivation $\mathcal{D}'$ for P' under $\mathcal{G}'$.*

To illustrate Session Fidelity in action, we can observe how the environment is constrained to follow the exact execution traces of the process it types.

Example 2.8 (Session Fidelity in Execution). Recall the process derivation from Example 2.4, initially typed by the hyperenvironment $\mathcal{G} = v : 1, w : \bot \mid y : 1$. The process first performs an observable input action on w. By session fidelity, this process transition is mirrored by the corresponding environment transition, consuming the resource typed by $\bot$:

$$\begin{aligned} v : 1, w : \bot \mid y : 1 &\xrightarrow{w()} v : 1 \mid y : 1 \\ &\xrightarrow{\tau} v : 1 \mid y : 1 \\ &\xrightarrow{y[]} v : 1. \end{aligned}$$

After the first transition, the process derivative is typed by $v : 1 \mid y : 1$. Viewed in isolation, the environment transition system admits an output action on y, since $y : 1$ is available as a resource. However, session fidelity does not state that every environment transition can be realised by the current process. Rather it states that each process transition has a corresponding environment transition. In the process term, the action $y[]$ remains guarded until an internal synchronisation has taken place. The environment therefore mirrors the intermediate τ-transition without changing its resources, and only afterwards mirrors the observable output on y by consuming the final $y : 1$ component.

Session fidelity is a central correctness property of a session-typed calculus: it expresses that process evolution is matched by the evolution of its typing resources. In the paper presentation, this correspondence is supported by implicit conventions for fresh names, bound variables, and structural rearrangement. In a proof assistant, these conventions must instead be represented explicitly and manipulated by proved lemmas. Establishing that types are consumed correctly therefore requires a concrete treatment of binding, opening, substitution, and freshness. The next chapter develops this binding architecture, which provides the foundation for the subsequent Lean formalisation and its metatheory.

2.8 Managing Bound Variables

Having established the operational semantics of πLL and the ultimate goal of verifying Session Fidelity, we must address the treatment of bound variables and freshness, a central technical challenge that underpins the entire formalisation.

As we have seen, many constructs in πLL introduce scoped identifiers. Restriction binds pairs of session endpoints, input constructs bind received channel names, polymorphic communication binds type variables, and duplication constructs introduce fresh channel endpoints. These bindings appear throughout both the process syntax and the labelled transition system.

The concrete choice of bound and introduced names is semantically irrelevant provided the binding structure is preserved. For example, renaming a restricted channel does not change the behaviour of a process. However, reasoning formally about substitutions, transitions, and typing derivations requires these names, scopes, and freshness conditions to be tracked explicitly.

This principle is formalised as *α-equivalence*, which identifies terms that differ only in the names of their bound variables. For instance, the following two processes are α-equivalent:

$$\nu ab\,(a[].\mathbf{0} \mid b().x[].\mathbf{0}) =_\alpha \nu zw\,(z[].\mathbf{0} \mid w().x[].\mathbf{0})$$

The operational semantics of πLL relies on α-equivalence using rule $=_\alpha$ to rename bound variables on the fly, ensuring that restricted scopes and input prefixes do not accidentally capture free names as processes evolve.

Substitution raises this exact issue. Consider the substitution where the free name x is replaced by z in the right-hand process from the previous example:

$$\nu zw\,(z[].\mathbf{0} \mid w().x[].\mathbf{0})$$

Here z is already bound by the restriction, so naively replacing x with z would cause the restriction to capture the newly inserted z. Before the substitution can proceed, and because restriction binds session endpoints as pairs, both the bound z and w must be α-renamed to fresh names, such as to a and b. Similarly, any binder within the process that would capture the substituted name must also be renamed before substitution continues.

In paper-based presentations this is commonly handled by *Barendregt's variable convention*, which assumes bound variables are always chosen distinct from free variables and one another whenever convenient. This makes proofs readable but remains an informal meta-level assumption.

A proof assistant requires every freshness condition and renaming step to be represented and verified explicitly [Aydemir et al. 2005]. A direct mechanisation using named variables therefore introduces substantial proof overhead for capture avoidance and α-equivalence reasoning. To avoid this, the formalisation adopts a *locally nameless* representation of binding, which eliminates α-equivalence as a proof obligation by construction. This is the subject of the following chapter.

3 Binding Architecture

This chapter develops the locally nameless binding architecture used throughout the formalisation. We begin by stating the design goals that motivate the approach (Section 3.1), then introduce the locally nameless representation and its core operations (Sections 3.2 and 3.3), and conclude with co-finite quantification (Section 3.4), which governs how inductive definitions interact with binders.

3.1 Design Goals and the Approach

This chapter introduces the binding representation used throughout the formalisation. The representation is motivated by three primary design goals:

(1) **Eliminate α-equivalence overhead.** α-equivalent terms should be represented identically, reducing α-equivalence reasoning to definitional equality within the proof assistant.
(2) **Preserve human readability.** Terms should remain explicit and concrete, keeping substitution and freshness reasoning close to their paper-based presentation.
(3) **Avoid informal meta-level conventions.** The representation should not rely on Barendregt's convention or unverified freshness assumptions. Capture avoidance and freshness conditions should instead be enforced by construction.

Although both channels and type variables involve binding, they exhibit fundamentally different operational behaviour. Session channels participate in dynamic communication and substitution across concurrent processes, while polymorphic type variables remain statically scoped within typing derivations. The formalisation therefore adopts different binding strategies for these two classes of identifiers while maintaining a common underlying binding architecture.

To achieve these goals, we adopt a locally nameless binding architecture for the calculus following Charguéraud [2012], separating free identifiers from bound occurrences. Bound variables are represented using indices, causing α-equivalent terms to coincide structurally, while free variables remain explicit and readable.

This separation simplifies substitution by restricting it to free variables while leaving bound indices untouched, thereby avoiding accidental capture by construction. As a result, many proofs that traditionally require substantial renaming infrastructure reduce instead to structural recursion and index manipulations.

The remainder of this chapter develops the theoretical machinery underlying this representation.

3.2 The Locally Nameless Representation

The locally nameless representation, as presented by Charguéraud [2012], separates variables into two disjoint syntactic categories: free names and bound indices. This combines the readability of named syntax for free variables with a structural representation of binding.

(1) **Bound variables** are represented using *De Bruijn indices* [de Bruijn 1972], where each index denotes the distance to its corresponding binder. Since the representation is purely positional, α-equivalent terms become structurally identical.
(2) **Free variables** retain explicit names, preserving the readability of open terms and allowing standard name-based reasoning about environments and transition labels.

By structurally separating these two concepts, the locally nameless approach simplifies standard operations such as capture-avoiding substitution. Since free variables and bound variables inhabit distinct syntactic categories, substitution on free variables is defined to not interact with bound indices. We apply this theory to both the structural and operational layers of πLL.

LN Syntax for Types. In the original presentation of πLL, polymorphic types rely on standard named variables. Under the locally nameless architecture, we expand the syntax of type variables to explicitly distinguish between free and bound occurrences. The grammar for types is updated as follows:

$$
\begin{array}{rcll}
T & ::= & \mathrm{free}(x) \mid \mathrm{bound}(k) & (\text{where } x, k \in \mathbb{N}) \\
A, B & ::= & \cdots \mid \exists.A \mid \forall.A \mid X \mid X^{\perp} & (\text{where } X \in T)
\end{array}
$$

Here, a free type variable carries a unique identifier x, while a bound type variable carries a De Bruijn index k. For example, the polymorphic type $\forall X.\exists Y.(X \parr Y)$ is structurally represented without names using the new syntax as $\forall.\exists.(\text{bound}(1) \parr \text{bound}(0))$, where 0 points to the inner existential binder and 1 points to the outer universal binder.

LN Syntax for Processes. Similarly, the standard process syntax displayed in Section 2.2 is updated to reflect the locally nameless separation of channel endpoints. A channel is split into its bound and free counterparts, where bound channel names are represented by indices, while free channels remain explicit identifiers. This split reflects a semantic distinction: free channel names are observable, appearing in transition labels and typing environments, while bound names are purely structural. Renaming a bound variable never changes the behaviour of a process, whereas renaming a free channel can alter which sessions it participates in. The locally nameless representation makes this distinction explicit by construction.

Naively removing the bound names from the process syntax would turn binding operations such as $x[y].P$ and $x(y).P$ into $x[].P$ and $x().P$ making them syntactically indistinguishable from their non-binding unit counterparts. To combat this we annotate anonymous binders using placeholder symbols. We use $\bullet$ for a bound process name, and $\diamond$ for a bound type variable.

$$
\begin{array}{rcll}
\mathit{Channel} & ::= & \text{free}(x) \mid \text{bound}(k) & (\text{where } x, k \in \mathbb{N}) \\
P, Q & ::= & \cdots \mid x[\bullet].P \mid x(\bullet).P \mid \nu(\bullet,\bullet)\, P \mid x(\diamond).P & (\text{where } x \in \mathit{Channel})
\end{array}
$$

While the visual anonymity of these constructors is evident from the grammar, their mechanical implementation relies on precise index management. For a multi-name binder like restriction, $\nu(\bullet,\bullet)\, P$, the two dual endpoints are simultaneously mapped to distinct indices within the continuation of P, namely 0 and 1. For communication prefixes like $x(\bullet).P$, the transmitted name is implicitly bound as index 0 within P, while the explicit free channel x is deliberately preserved as the subject. By maintaining explicit free names strictly for these outward-facing communications, the transition labels and operational semantics of the calculus remain completely unaffected by the architectural shift.

While the locally nameless architecture elegantly resolves α-equivalence, moving between abstracted and instantiated forms requires specialised operations. To execute a substitution or evaluate a process across a binder, we must formally define how to open and close these terms.

3.3 Opening and Closing Binders

Because the locally nameless representation separates free and bound variables, standard substitution cannot be applied uniformly under binders. Instead, moving between bound and free representations is handled by two dual operations, namely *opening* and *closing* [Charguéraud 2012].

Opening (Instantiation). Opening is the operation of traversing into a binding construct and replacing the exposed bound index with a concrete free name. Formally, opening a process P at depth k with a free name x is denoted $\{k \mapsto x\}P$.

The operation proceeds via structural recursion. A depth counter, initially set to 0, tracks the number of binders traversed. As the recursion descends through the syntax tree, the counter increments when passing a new binder. Consider being at depth k, then traversing a single-name binder like $x(\bullet).P$ increments the depth to $k+1$, while traversing a double-name binder like $\nu(\bullet,\bullet)\, P$ would increment it to $k+2$.

When the recursion reaches a bound variable $\text{bound}(n)$, the index n is compared against the current depth k. If $n = k$, the variable refers to the binder currently being opened and is replaced by the free name $\text{free}(x)$. Otherwise, the index refers to a different binder and is left unchanged.

Crucially, when the opening operation encounters an existing free variable $\mathrm{free}(y)$, it simply ignores it. This guarantees that opening is entirely capture-avoiding by construction. In the operational semantics, when a single binder like an input prefix $z(\bullet).P$ receives a channel x, the continuation P is opened as $\{0 \mapsto x\}P$. For a restriction $\nu(\bullet, \bullet)\,P$, the dual endpoints x and y are opened sequentially as $\{0 \mapsto x\}(\{1 \mapsto y\}P)$.

Closing (Abstraction). Closing is the companion abstraction operation to opening. It is the operation of taking an open term and abstracting a specific free name x into a bound index at depth k, denoted $\{k \hookleftarrow x\}P$. While recursing, bound indices are ignored as they cannot contain free names. When the recursion encounters a free term like $\mathrm{free}(y)$, it checked if $y = x$. If they match, the free name is replaced by $\mathrm{bound}(k)$.

This operation is primarily used when constructing new binders, such as moving a free channel into a restricted scope. To close multiple free names simultaneously, such as abstracting x and y into a restriction, the operations are applied sequentially, by first closing y at depth $k + 1$, followed by closing x at depth k.

Locally Closed Terms. The separation of names and indices introduces a new syntactic anomaly, namely that a term can contain a *dangling* index that does not point to any enclosing binder. For example, the process term $\mathrm{bound}(0)[\,].\mathbf{0}$ is syntactically valid, matching the syntax for an empty send, but mathematically it is meaningless because the index 0 has no corresponding binder.

To filter out these meaningless terms, the locally nameless architecture introduces the predicate of being *locally closed*, denoted $lc(P)$. A term is locally closed if and only if all of its De Bruijn indices are properly bound by an enclosing constructor.

The locally closed predicate excludes terms containing dangling indices and therefore characterises the well-formed syntax of the calculus. Establishing this predicate is not only a static validation step, but rather it forms a key invariant throughout the mechanisation, which guarantees the soundness of the calculus. Generally, any subsequent definition, such as variable substitution, structural congruence, or operational semantics, must be mathematically proven to *preserve* the locally closed property. In practice, this is achieved through the *regularity of typing*, which requires proving that all valid typing derivations inherently produce locally closed terms. Consequently, operations proven to preserve typing will naturally carry this invariant as a structural guarantee.

3.4 Co-finite Quantification

With the syntax partitioned into names and indices, and the mechanics of opening and closing established, the final theoretical hurdle is defining how inductive relations, such as typing judgements and labelled transitions, operate across binders.

In paper-based proofs, when a rule requires opening a binder, Barendregt's variable convention allows the author to simply state: *pick a fresh name* x. In a proof assistant, this informal generation of a fresh name must be encoded into the constructors of the inductive relations. Two naive approaches to bind this fresh name, both of which introduce severe mechanical flaws [Aydemir et al. 2008; Charguéraud 2012]:

In paper-based proofs, when a rule requires opening a binder, Barendregt's Variable Convention allows the author simply to state: *pick a fresh name* x. In a proof assistant, this informal choice of a fresh name must instead be encoded in the constructors of the inductive relations. Two naive approaches to representing this choice introduce significant mechanisation difficulties [Aydemir et al. 2008; Charguéraud 2012]:

(1) **Existential Quantification ($\exists x \notin \mathrm{f}(P)$):** The rule requires the property to hold for at least one fresh name. While this makes proof construction straightforward, it yields weak

induction hypotheses, since induction only provides the property for one particular fresh name chosen during the original derivation.

(2) **Universal Quantification ($\forall x$):** The rule requires the property to hold for every name. This provides a strong induction hypothesis, but is too restrictive for proof construction, since the property cannot generally be shown for names that already occur free in the surrounding context.

Following Aydemir et al. [2008]; Charguéraud [2012], the locally nameless architecture adresses this dillemma using co-finite quantification. Rather than quantifying over one name or all names, the inductive rules quantifies universally over all names outside an arbitrary, finite set L. Formally, a rule requiring a fresh name uses the premise: $\forall x \notin L$.

By deferring the exact choice of the forbidden set L to the user at the time of proof, co-finite quantification achieves the best of both worlds. When constructing a proof tree, the user can instantiate L as the set of all free variables currently in the environment, process, and transition labels ($\mathrm{f}(P) \cup \mathrm{f}(\Gamma) \cup \ldots$). This ensures that any name quantified over by the premise satisfies the required freshness assumptions. Conversely, when performing structural induction, the co-finite universal quantifier guarantees that the induction hypothesis applies to *any* fresh name the user could possibly need, provided they pick one outside the finite set L.

Application to πLL Types. Although type variables are represented using the locally nameless separation of free and bound constructors, their metatheory is handled using depth-indexed reasoning rather than cofinite quantification. Because type variables are static and do not dynamically communicate across parallel scopes like channels do, managing freshness sets L for them introduces unnecessary proof overhead.

Instead, the system evaluates polymorphic typing rules using a purely depth-indexed approach, using a De Bruijn shifting operator. We define the shifting of a type A at a cutoff depth d by distance c, denoted mathematically as $\uparrow_d^c A$. Using this infrastructure, consider the typing rule for the universal type input ($\forall.B$):

$$\frac{n+1 \vdash P :: \uparrow_0^1 \Gamma, x : B}{n \vdash x(\diamond).P :: \Gamma, x : \forall.B}\ \forall_{\mathrm{DB}}$$

Rather than opening the binder with a fresh explicit name, the typing judgment simply increments a depth counter $n+1$, legally bringing index 0 into scope. To maintain correct index distances, the typing environment Γ is explicitly shifted by a distance of 1 at cutoff 0 ($\uparrow_0^1 \Gamma$), incrementing all free type indices within the typing environment to prevent accidental capture.

Application to πLL Channels. This co-finite quantification strategy fundamentally reshapes the inductive definitions for process channels. For example, consider the typing rule that introduces the prefix $x(\bullet).P$. Under a paper-based presentation, the premise implicitly requires a fresh session name y. Under our mechanised architecture, the rule uses co-finite quantification and the opening operation:

$$\frac{\forall y \notin L, \quad n \vdash \{0 \mapsto y\}P :: \Gamma, x : B, y : A}{n \vdash x(\bullet).P :: \Gamma, x : A \parr B}\ \parr_{\mathrm{LN}}$$

Here, the rule states that the process is well-typed if, for any fresh channel variable y outside some finite set of forbidden variables L, the opened process P is well-typed under the environment extended with the opened name. By utilizing this mixed evaluation strategy, co-finite quantification for dynamic channels, and depth-indexed evaluation for static types, the architecture minimizes bureaucratic renaming overhead while maintaining a unified underlying syntax.

This mixed strategy also determines where the remaining proof overhead appears. Depth-indexed treatment of polymorphic types avoids managing fresh type-variable sets, but requires explicit shifting and weakening lemmas when typing derivations cross polymorphic binders. Co-finite quantification for process channels avoids α-equivalence reasoning and provides strong induction hypotheses, but later requires bridging lemmas when an operational transition chooses concrete fresh names. These costs are reflected in the mechanised definitions and proofs developed in the following chapters.

With this mathematical foundation established, the conceptual theory of πLL is fully prepared for implementation. The following chapter details how this locally nameless architecture is concretely mechanised in the Lean 4 proof assistant.

4 Formalisation

This section describes the mechanisation of πLL in Lean 4, bridging the theoretical foundations developed in Sections 2 and 3 and their concrete implementation. The formalisation is organised into three top-level namespaces: `PiLL.Model`, which defines the static structure of the calculus, `PiLL.Semantics`, which defines its operational behaviour, and `PiLL.SessionFidelity`, which establishes the metatheoretic results connecting the two. Each component is presented in dependency order, highlighting design decisions and divergences from the presentation of Montesi and Peressotti [2021].

4.1 Model

The `Model` namespace formalizes the static structure of the full πLL calculus and is organized around the layered architecture introduced in Section 2. The mechanisation proceeds bottom-up: session types, defined in `STypes/`, provide the binding structure used by processes in `Processes/`, processes are then typed relative to environments and hyperenvironments from `Environments/` and `HyperEnvironment/`, finally, typing derivations and their metatheoretic infrastructure are collected in `Judgement/`. This includes inversion lemmas, linearity properties, substitution principles, and the co-finite freshness machinery described in Section 3.4. Shared syntactic infrastructure used across these layers is provided by `Base.lean`. An overveiw of the file structure can be found in Listing 2.

4.1.1 Session Types. Session types are defined in `STypes/Basic.lean` as the inductive datatype `Types`. Following the locally nameless syntax established in Section 3.2, type variables are isolated into a separate inductive type, `TVar`, which explicitly distinguishes between free and bound occurrences:

```
inductive TVar : Type
  | free  (x : FTVar)
  | bound (x : BTVar)
```

Here, `FTVar` represents a free type variable identified by a natural number, while `BTVar` represents a bound type variable encoded as a De Bruijn index.

The remaining constructors directly mirror the theoretical grammar of the calculus. In addition, the formalisation introduces uninterpreted atomic types (`atom` and `atomDual`), representing protocol leaves not further decomposed by the session type grammar and providing the base cases for the inductive structure of the syntax. Table 3 illustrates a representative correspondence between the paper syntax, the locally nameless representation, and the Lean implementation.

Notably, the polymorphic binders $\forall.A$ and $\exists.A$ reflect the depth-indexed approach detailed in Section 3.4. As shown in Section D.1, the corresponding Lean constructors keep the bound De

Table 3. Comparison of standard, locally nameless, and Lean representations for πLL types.

Standard	Locally Nameless	Lean Constructor	Binds
$A \otimes B$	$A \otimes B$	`| tensor (A B : Types)`	None
1	1	`| one`	None
$\forall X.A$	$\forall.A$	`| forall_ (A : Types)`	1 type
$\exists X.A$	$\exists.A$	`| exists_ (A : Types)`	1 type
X	X	`| var (v : TVar)`	None

Bruijn index implicit, taking only the continuation body `A` as an argument. An underscore "_" is appended to the constructor names to prevent namespace collisions with Lean's built-in `forall` and `exists` keywords.

Duality. `Types.dual` implements duality as a structurally recursive function, mapping each constructor to its dual. For example:

```
def Types.dual : Types -> Types
  | .atom n     => .atomDual n
  | .atomDual n => .atom n
  | .tensor A B  => .parr    (dual A) (dual B)
  | .forall_ A   => .exists_ (dual A)
  | .one         => .bot
  | ...
```

The fundamental property that duality is an involution, $(A^{\perp})^{\perp} = A$, is proved in `STypes/Lemmas.lean` by structural induction.

The formalisation additionally establishes that duality preserves local closure and commutes with both shifting and substitution:

$$A.\mathtt{lc} \iff A^{\perp}.\mathtt{lc} \qquad \uparrow^{c}_{d} (A^{\perp}) = (\uparrow^{c}_{d} A)^{\perp} \qquad B^{\perp}\{A/k\} = (B\{A/k\})^{\perp}$$

Local closure. `STypes/Properties.lean` defines local closure for types as a depth-indexed predicate `Types.lc (k : Nat) : Types → Prop`, where the depth parameter k tracks how many binders the recursion has traversed.

At the leaves of the syntax tree, local closure is evaluated by the helper predicate `TVar.lc`. Because free variables are explicit they do not contain any indices, as such they are inherently locally closed at any depth. By contrast, a bound variable is only locally closed if its De Bruijn index i is strictly less than the current depth k, ensuring it points to an enclosing scope:

```
def TVar.lc : Nat -> TVar -> Prop
  | _, .free _    => True
  | k, .bound i   => i < k
```

The primary `Types.lc` predicate is then defined by structural recursion. For non-binding constructors, the depth parameter distributes across the sub-terms. When a variable is encountered, the predicate delegates the check to `TVar.lc`. When descending into the body of a polymorphic binder, the depth counter increments by one, bringing the newly introduced index safely into scope:

```
def Types.lc : Nat -> Types -> Prop
  | _, .one          => True
```

```
  | k, .var v        => TVar.lc k v
  | k, .tensor A B   => A.lc k ∧ B.lc k
  | k, .forall_ A    => A.lc (k + 1)
  | k, .exists_ A    => A.lc (k + 1)
  | ...
```

Finally, a type is defined as *closed* if it contains no dangling indices at the top level, meaning it is locally closed at depth 0. This is registered as the abbreviation `Types.lc_0`.

Shifting. `STypes/Shift.lean` defines shifting following the standard De Bruijn shifting operation introduced in Section 3.4. The operation traverses the syntax tree structurally using a cutoff depth d and a shift amount c.

For standard connectives, shifting distributes recursively across subterms, while base constructors such as one and `bot` remain unchanged. When traversing a polymorphic binder like `forall_`, the cutoff depth increments by one to account for the newly entered scope:

```
def Types.shift (d c : Nat) : Types -> Types
  | .var v        => .var (v.shift d c)
  | .tensor A B   => .tensor (A.shift d c) (B.shift d c)
  | .forall_ A    => .forall_ (A.shift (d + 1) c)
  | t             => t
```

At the variable level, free variables remain unchanged. For bound indices, the function compares the De Bruijn index i against the cutoff depth d. Indices satisfying $i < d$ are locally bound within the traversed scope and therefore remain unchanged. Otherwise, the index is incremented by c to preserve its reference to the surrounding environment:

```
def TVar.shift (d c : Nat) : TVar -> TVar
  | .bound i => if i < d then .bound i else .bound (i + c)
  | .free n  => .free n
```

The operation is exposed through the common `HasShift` interface, allowing the notation $\uparrow^c_d A$, $A \uparrow c$ and A^+ to be used uniformly for types and, later, for processes, environments, and hyperenvironments; the shared notation declarations are given in Listing 3.

Substitution. `STypes/Subst.lean` defines substitution by replacing a target De Bruijn index k in a host type B with a type A,denoted $B\{A//k\}$. At the variable level, the function compares a bound De Bruijn index i with the target depth k. If $i = k$, the target index has been reached and the variable is replaced by A. If $i < k$, the variable is bound by an inner scope and is left unchanged. Finally, if $i > k$, the index refers to an outer binder beyond the substituted variable. Since substitution removes the binder corresponding to k, such indices are decremented by one to preserve their relative distance to the surrounding environment.

```
def Types.subst (B A : Types) (k : Nat) : Types :=
  match B with
  | .var (.free n)  => .var (.free n)
  | .var (.bound i) =>
      if i == k then A
      else if i > k then .var (.bound (i - 1))
      else .var (.bound i)
  | .tensor A B => .tensor (A.subst A k) (B.subst A k)
```

The operation is lifted to the full Types syntax by structural recursion. For ordinary connectives, substitution distributes across subterms. When descending into a polymorphic binder such as `forall_`, the target depth is incremented to $k + 1$, and the substituting type A is shifted by one. This prevents the free indices in A from being captured by the newly entered binder:

```
| .forall_ B => .forall_ (B.subst (shift 0 1 A) (k + 1))
```

The interaction between substitution and local closure is captured by `Types.lc_subst_inv`, which states that substituting a type A locally closed at depth $n + k$ for index k in B preserves local closure: $(B\{A/k\}).\mathtt{lc}(n + k) \iff B.\mathtt{lc}(n + k + 1)$. Intuitively, substituting for index k removes one binder from B, so local closure at $n + k$ in the substituted term corresponds to local closure at $n + k + 1$ in the original. The specialisation `Types.lc_subst_inv_0` fixes $k = 0$, which is the form later used in the proof of `Typing_preserves_lc`.

As with shifting, substitution is exposed through the common `HasSubst` interface, allowing the notation $B\{A//k\}$ to be reused when substitution is lifted to processes, environments, and hyperenvironments (see Listing 3).

Notation. `STypes/Notation.lean` introduces notation mirroring the mathematical presentation from Section 2.3. The only divergence is that exponentials are written $!!A$ and $??A$ to avoid clashes with Lean syntax. Linear implication $A \multimap B$ is defined as syntactic sugar for $A^{\perp} \parr B$. The corresponding Lean notation declarations are given in Listing 4.

4.1.2 Processes. Processes are defined in `Processes/Basic.lean` as the inductive type `Proc`. Following the established locally nameless architecture, channel references are partitioned into a dedicated `Channel` inductive. Explicit free channels are instantiated as concrete natural numbers (`FPName`). While the raw process syntax permits arbitrary reuse of these numbers, using `Nat` allows the formalisation to systematically compute strictly fresh names by incrementing the supremum of any finite set of existing names (mechanised in `Processes/Fresh.lean`). Conversely, bound channels are represented using De Bruijn indices, encoding their relative distance to the corresponding binder.

```
inductive Channel : Type where
  | free  (x : FPName)
  | bound (x : BPName)
```

As discussed in Section 3.2, binding constructs in the process syntax no longer carry explicit bound channel names. Instead, bound occurrences are captured purely by their relative position. Comparing the standard paper grammar with the intermediate locally nameless notation and the final constructors of the `inductive Proc` type illustrates this abstraction:

Standard	**Locally Nameless**	**Lean Constructor**	**Binds**
$x[y].P$	$x[\bullet].P$	`| tensor (x : Channel) (P : Proc)`	1 channel
$x(y).P$	$x(\bullet).P$	`| parr (x : Channel) (P : Proc)`	1 channel
$\nu xy\,P$	$\nu(\bullet,\bullet)\,P$	`| cut (P : Proc)`	2 channels
$x(X).P$	$x(\diamond).P$	`| input (x : Channel) (P : Proc)`	1 type
$x[\textsc{dup}](x').P$	$x[\textsc{dup}](\bullet).P$	`| duplicate (x : Channel) (P : Proc)`	1 channel

Notice that `cut` takes only a continuation P with no explicit channel arguments, as both dual endpoints are simultaneously bound as indices 0 and 1 within the scope of P. Similarly, for constructs like `tensor`, `parr`, and `input`, the subject channel x over which the interaction occurs is kept explicit, while the name or type variable bound in the continuation is anonymized and represented by an index. The full inductive definition is listed in Section D.2.

Opening and closing. `Processes/OpenClose.lean` defines opening and closing operations following the locally nameless principles established in Section 3.3. In the theoretical presentation, opening a process at depth k with x is denoted as $\{k \mapsto x\}P$, and conversely, abstracting a specific name back into a bound index is denoted as $\{k \hookleftarrow x\}P$.

To mechanise this in Lean, we denote explicit free channels as $\#x$ (mapping to `Channel.free x`) and introduce custom notation for these substitution operations. Expressions such as $P(\!(k|\#x)\!)$ and $P(\!(k|\#x, \#y)\!)$ denote the opening of a process by instantiating the k-th (and $k+1$-th) bound indices with free channels. Conversely, $P\langle\!\langle k|\#x\rangle\!\rangle$ and $P\langle\!\langle k|\#x, \#y\rangle\!\rangle$ denote the closing of a process by abstracting x and y back into De Bruijn indices. For convenience, the depth parameter can be omitted when operating at the innermost binder ($k = 0$), simplifying the notation to $P(\!(\#x, \#y)\!)$ and $P\langle\!\langle \#x, \#y\rangle\!\rangle$. The typeclasses and notation declarations supporting these overloaded operations are collected in Section C.

Because binders are represented positionally, the depth parameter k increments according to the number of channels bound by each constructor. For standard communication prefixes like `tensor`, `parr`, and the server `duplicate` operation, exactly one channel is bound, incrementing k by one. The restriction constructor (`cut`) simultaneously binds two dual endpoints, requiring k to increment by two. The `input` constructor binds a type variable rather than a channel and therefore leaves k unchanged. The `server` constructor additionally carries an explicit set of dependency channels `zs : Finset Channel`, representing the replicable processes the server depends on,which must be acted on simultaneously with x and P.

```
def Proc.open (P : Proc) (u : Channel) (k : Nat) : Proc :=
  match P with
  | .tensor x P     => .tensor (x.open u k) (P.open u (k + 1))
  | .parr   x P     => .parr   (x.open u k) (P.open u (k + 1))
  | .cut      P     => .cut                 (P.open u (k + 2))
  | .duplicate x P  => .duplicate (x.open u k) (P.open u (k + 1))
  | .server x zs P  => .server (x.open u k) (zs.open u k) (P.open u k)
  | .input  x P     => .input  (x.open u k) (P.open u k)
  | ...
```

For channel occurrences, the recursive traversal delegates to `Channel.open`, for the dependency set carried by a server, it uses the pointwise lifting `Finset.open`. For bound channels, the De Bruijn index i is compared against the target depth k and replaced with v upon an exact match. For dependency sets, `Channel.open` is mapped pointwise. In both cases free channels remain entirely unaffected:

```
def Channel.open (u v : Channel) (k : Nat) : Channel :=
  match u with
  | bound i => if i == k then v else bound i

def Finset.open (zs : Finset Channel) (v : Channel) (k : Nat) : Finset Channel :=
  zs.image (fun u => u.open v k)
```

For restriction, the two-name opening interface is instantiated by sequential opening, replacing index $k+1$ with the second endpoint v, then replacing index k with the first endpoint u:

```
instance : HasOpenTwo Proc Channel Channel Nat where
  open_ P u v k := (Proc.open (Proc.open P v (k + 1)) u k)
```

To complete the bidirectional infrastructure, `Proc.close`, `Channel.close`, and `Finset.close` define the corresponding closing operation. `Proc.close` traverses the process tree using a structural recursion identical to `Proc.open`, applying the same depth-incrementing rules. At the leaves, `Channel.close` compares each free channel against the target name x and abstracts it into a bound De Bruijn index at depth k upon a match, while bound indices are left unchanged. Dependency sets are handled pointwise by `Finset.close`:

```
def Channel.close (u : Channel) (x : FPName) (k : Nat) : Channel :=
  match u with
  | .free name => if x == name then .bound k else .free name
  | .bound i   => .bound i

def Finset.close (zs : Finset Channel) (x : FPName) (k : Nat) : Finset Channel :=
  zs.image (fun u => u.close x k)
```

Local closure. `Processes/LocalClosure.lean` defines local closure for processes as a two-parameter predicate `Proc.lc (k n : Nat) : Proc → Prop`. Because the calculus features a two-sorted syntax, the predicate must maintain independent depth counters: k tracks the depth of channel binders, while n tracks the depth of type binders. The latter is required because processes may contain embedded types whose free indices must be interpreted relative to the surrounding type-binder depth.

The structural recursion evaluates each constructor by incrementing the appropriate depth counter according to the number and sort of binders introduced. Channel binders increase the channel depth k, while type binders increase the type depth n. For example, `tensor` binds a single continuation channel, incrementing the depth to $k + 1$, whereas `cut` binds two dual endpoints simultaneously, incrementing the depth to $k + 2$. Conversely, `input` binds a type variable, leaving k unchanged while incrementing n to $n + 1$. The `server` constructor therefore requires local closure of x, zs, and P. Constructors introducing no binders, leave both counters unchanged

```
def Proc.lc : Nat -> Nat -> Proc -> Prop
  | _, _, .nil             => True
  | k, n, .one x P         => x.lc k ∧ P.lc k n
  | k, n, .tensor x P      => x.lc k ∧ P.lc (k + 1) n
  | k, n, .cut P           => P.lc (k + 2) n
  | k, n, .input x P       => x.lc k ∧ P.lc k (n + 1)
  | k, n, .duplicate x P   => x.lc k ∧ P.lc (k + 1) n
  | ...
```

At the leaves, `Channel.lc` checks that any bound index is strictly within the current depth, while free channels are trivially locally closed. For dependency sets, `Finset.lc` requires every channel in the set to satisfy `Channel.lc k`:

```
def Channel.lc : Nat -> Channel -> Prop
  | _, .free _   => True
  | k, .bound i  => i < k

def Finset.lc (zs : Finset Channel) (k : Nat) : Prop :=
  ∀ z ∈ zs, z.lc k
```

A process is considered globally *closed* when it contains no dangling indices of either sort, meaning `Proc.lc 0 0` holds. This is registered as the abbreviation `Proc.lc_0`.

Free names and freshness. `Processes/Names.lean` defines `Proc.f : Proc → Finset FPName` to compute the free channel names of a process. The function traverses the syntax tree structurally, collecting all explicit free channel identifiers into a finite set. Since locally nameless binders are represented using anonymous indices, bound channels contribute nothing to this set.

For constructors carrying explicit subject channels, such as `tensor` and `link`, the free names of those channels are combined with the recursively computed free names of their continuations. Constructors without explicit channel identifiers of their own, such as `cut`, contribute only the free names recursively obtained from their continuations. The `server` constructor contributes the free names of its subject channel, its dependency set and continuation:

```
def Proc.f : Proc -> Finset FPName
  | .cut P          => P.f
  | .par P Q        => P.f ∪ Q.f
  | .link x y       => x.f ∪ y.f
  | .tensor x P     => x.f ∪ P.f
  | .one x P        => x.f ∪ P.f
  | .server x zs P  => x.f ∪ zs.f ∪ P.f
  | .nil        => {}
  | ...
```

At the leaves, `Channel.f` returns the singleton set for free channels and the empty set for bound indices. For dependency sets, `Finset.f` collects free names across all channels in the set via `biUnion`:

```
def Channel.f : Channel -> Finset FPName
  | .free x   => {x}
  | .bound _ => {}

def Finset.f (zs : Finset Channel) : Finset FPName :=
  zs.biUnion Channel.f
```

The resulting finite set is used by `Processes/Fresh.lean` to generate names outside a forbidden context. Since free process names are natural numbers, a fresh name for a finite set L is computed as `freshName L` $= \sup(L) + 1$, with `fresh_is_fresh` confirming that every element of L is strictly smaller than this value. The lemmas `exists_one_fresh` and `exists_two_fresh` then guarantee one or two distinct names outside L, providing witnesses for the co-finite quantification premises used throughout the typing and transition rules, in particular for restriction, where two fresh and distinct endpoints must be introduced simultaneously.

Substitution. The process syntax supports two substitution operations, reflecting its two-sorted binding structure. Name substitution is defined in `Processes/SubstNames.lean` as the function `Proc.substNames (R T : FPName) : Proc → Proc`, replacing all explicit free occurrences of the target name T with the replacement name R. Since bound channels are represented by De Bruijn indices, name substitution only acts on explicit free channel leaves and leaves binders untouched. Consequently, the operation is capture-avoiding by construction and does not require α-renaming or additional freshness side conditions.

Name substitution is defined in `Processes/SubstNames.lean` as `Proc.substNames (R T : FPName) : Proc → Proc`, replacing all explicit free occurrences of T with R. At the leaves, `Channel.subst` replaces a matching free channel and leaves bound indices unchanged; dependency sets are handled pointwise by `Finset.subst`:

```
def Channel.subst (R T : FPName) : Channel -> Channel
```

```
  | .free x  => if x == T then .free R else .free x
  | .bound i => .bound i

def Finset.subst (zs : Finset Channel) (R T : FPName) : Finset Channel :=
  zs.image (fun u => u.subst R T)
```

Type substitution is defined in `Processes/SubstTypes.lean` as `Proc.substTypes (A : Types) (k : Nat) : Proc → Proc`, substituting A for the De Bruijn type index k throughout a process. Whenever the traversal encounters an embedded type, such as the type argument of an `output` prefix, it delegates to the type-level substitution operation `Types.subst`. When crossing an `input` prefix, which binds a type variable, the target depth is incremented and the substituted type is shifted, mirroring the binder case of `Types.subst` from Section 4.1.1:

```
def Proc.substTypes (A : Types) (k : Nat) : Proc -> Proc
  | .input x P => .input x (P.substTypes (A.shift 0 1) (k + 1))
```

A consequence of this representation is that no analogous channel shifting operation is required. In a pure De Bruijn encoding, variables are represented as relative indices and must be shifted when crossing binders. Free channels, however, are represented as global identifiers rather than relative positions, so their representation is unaffected by binder depth. This avoids structural adjustment of channel names during substitution and simplifies the process-level infrastructure.

Type shifting. `Processes/SubstTypes.lean` also defines `Proc.shiftTypes (d c : Nat) : Proc → Proc`, which lifts the type-level shifting operation to processes. The traversal is uniform across most constructors, delegating to `Types.shift` whenever an embedded type is encountered, such as the type argument of an `output` prefix. The only non-trivial case is `input`, which binds a type variable and therefore increments the cutoff depth d before traversing its continuation, mirroring the binder case of `Types.shift`:

```
def Proc.shiftTypes (d c : Nat) : Proc -> Proc
  | .output x P A  => .output x (P.shiftTypes d c) (A.shift d c)
  | .input x P     => .input x (P.shiftTypes (d + 1) c)
  | ...
```

Structural Properties. `Processes/Lemmas.lean` establishes the fundamental structural properties required by the later typing and operational metatheory. The lemmas fall into three groups.

The first group supports the typing invariants. The lemmas `Proc.f_open_erase` and `Proc.f_open_two_erase` establish that opening a process with fresh names and then erasing those names from its free-name set recovers the original free names. They are used in the proof of `Typing_f_eq_names`. The lemmas `Proc.lc_of_open_gen` and `Proc.lc_of_open_two` show that opening a locally closed abstraction with locally closed names yields a locally closed process. They therefore support the proof of `Typing_preserves_lc`, which establishes that well-typed processes are locally closed.

The second group supports the substitution lemmas. The lemmas `Proc.open_substNames_comm` and `Proc.openCut_substNames_comm` establish that opening commutes with name substitution. Their counterparts for type substitution, `Proc.open_substTypes_comm` and `Proc.openCut_substTypes_comm`, establish the analogous property for substitution of types under binders. These results are used in the typability proof, where co-finite witnesses from the typing rules must be reconciled with the concrete names introduced by process transitions.

The third group supports freshness reasoning. `Proc.not_mem_f_open` states that a name fresh for both a process and a channel does not appear in the opened process, providing the freshness conditions needed in the typability cases. These results provide the core infrastructure required for the transition semantics developed in subsequent chapters.

Notation. `Processes/Notation.lean` introduces process prefix notation mirroring the locally nameless syntax established in Section 3.2. For example, `#x`⟦•⟧`.P` maps to `Proc.tensor (#x) P` and $\nu(\bullet, \bullet)$`P` maps to `Proc.cut P`. Parallel composition and the terminated process are written $P \mid Q$ and $\mathbf{0}$ respectively. The full notation is listed in Listing 5.

4.1.3 Environments. Environments are defined in `Environment/Basic.lean` as `Env`, an abbreviation for a list of channel-type pairs:

```
abbrev Elem := (FPName × Types)
abbrev Env  := List Elem
```

The choice of `List` as the underlying representation deserves explanation. Theoretical environments are unordered and duplicate-free, suggesting `Finset` as a natural implementation. However, since `Finset` is implemented as a quotient over lists, it complicates structural induction, pattern matching, and inversion over environments. This proved inconvenient in a development where typing derivations are frequently inverted and typing environments must be rearranged explicitly. A second attempt using association lists (`AList`) encountered similar proof-engineering difficulties. The final design therefore uses plain lists and recovers the unordered structure through an explicit permutation relation, trading quotient reasoning for structural induction and simpler interaction with Lean's tactic machinery.

Recovering the PCM structure. Merging two environments is Environment composition is implemented as list concatenation, yielding a simple total merge operation:

```
abbrev Env.merge (Γ Δ : Env) : Env := Γ ++ Δ
infixl:69 "," => Env.merge
```

Because Lean already reserves the ordinary comma for syntactic purposes, the implementation uses a visually similar comma-like symbol for environment merge. In the thesis presentation, this is rendered as the conventional comma notation for typing environments.

This representation does not make environments a partial commutative monoid by definition. List concatenation is total and order-sensitive, meaning that associativity and identity are inherited from append, while commutativity must be recovered up to permutation. The formalisation therefore uses Lean's built-in `List.Perm` relation, exposed through the common `HasPerm` interface and written $\Gamma \sim \Delta$. The corresponding monoidal laws include:

```
lemma Env.merge_unitL (Γ : Env) : ∅,Γ = Γ
lemma Env.merge_unitR (Γ : Env) : Γ,∅ = Γ
lemma Env.merge_assoc (Γ Δ Ξ : Env) : Γ,Δ,Ξ = Γ,Δ,Ξ
lemma Env.merge_comm (Γ Δ : Env) : Γ,Δ ~ Δ,Γ
```

The partiality condition is enforced separately by predicates on the merged resources. The function `Env.names` extracts the domain of an environment as a finite set of channel names. Mutual compatibility is expressed by disjointness of these domains, while internal linearity is expressed by the absence of duplicate names:

```
def Env.names (Γ : Env) : Finset FPName := (Γ.map Prod.fst).toFinset
def Env.disjoint (Γ Δ : Env) : Prop := Disjoint Γ.names Δ.names
def Env.Nodup (Γ : Env) : Prop := (Γ.map Prod.fst).Nodup
```

The connection between these predicates and merge is captured by `Env.Nodup_merge_iff`, which states that a merged environment is linear exactly when both component environments are individually linear and their domains are disjoint:

```
lemma Env.Nodup_merge_iff {Γ Δ : Env} :
  (Γ,Δ).Nodup ↔ (Γ.Nodup ∧ Δ.Nodup ∧ Γ.disjoint Δ) := by ...
```

The proof reduces directly to the corresponding property of list append, `List.nodup_append`, together with the finite-set formulation of environment-domain disjointness.

Server usability. The typing rule for server replication requires all dependency channels to carry exponential request types. This side condition is captured by `Env.serverUsable`, which asserts that every type in an environment satisfies `Types.isServerUsable`, i.e. has the form $?B$.

```
def Env.serverUsable (Γ : Env) : Prop :=
  ∀ p, p ∈ Γ -> p.snd.isServerUsable
prefix:max "?ₑ" => Env.serverUsable
```

Lifted operations. Since environments contain types, the locally nameless operations from Section 4.1.1 are lifted pointwise over the type component of each `Elem`. Local closure, type shifting, name substitution, and type substitution are all defined by mapping over the list. The file `Environment/Lemmas.lean` proves that these operations preserve the names, disjointness, nodup, and permutation properties of environments.

The interaction between shifting and local closure is captured by `Env.lc_shift_inv`, which states that shifting an environment by one preserves local closure: $(\Gamma^{+}).\mathtt{lc}(n+k+1) \iff \Gamma.\mathtt{lc}(n+k)$. Shifting increments all free type indices, so an environment locally closed at depth $n+k$ becomes locally closed at depth $n+k+1$ after shifting, and vice versa. The specialisation `Env.lc_shift_inv_0` is used in the `forall_` case of `Typing_preserves_lc` to recover local closure of the unshifted environment from that of the shifted one.

4.1.4 Hyperenvironments. Hyperenvironments are defined in `HyperEnvironment/Basic.lean` as `HyperEnv`, an abbreviation for a list of environments:

```
abbrev HyperEnv := List Env
abbrev HyperEnv.merge (𝒢 ℋ : HyperEnv) : HyperEnv := 𝒢 ++ ℋ
infixl:55 " |ₕ " => HyperEnv.merge
```

As for environments, merging is implemented as list concatenation, using $\varnothing$ as the unit and $\mathcal{G}\,|_h\,\mathcal{H}$ as notation for composition. Associativity and unit laws therefore follow from the underlying list structure. However, recovering the commutative structure requires more than the ordinary list permutation relation, since hyperenvironments are lists whose elements are themselves environments considered modulo permutation.

Deep permutation. For environments, Lean's `List.Perm` relation is sufficient, since it allows arbitrary reordering of channel-type pairs. For hyperenvironments, however, permutation must operate at two levels: it must allow reordering of component environments, and it must also respect permutation equivalence inside each component environment.

For example, consider the hyperenvironment containing a single component environment $[x : A, y : B]$. Ordinary `List.Perm` treats this component environment as an opaque element, so it does not identify it with $[y : B, x : A]$. The custom relation `HyperEnv.Perm` instead permits component environments to be replaced by permutation-equivalent environments.

```
inductive HyperEnv.Perm : HyperEnv -> HyperEnv -> Prop where
  | nil  : Perm [] []
  | cons : {Γ Δ : Env} {𝒢 ℋ : HyperEnv} : Γ ~ Δ -> Perm 𝒢 ℋ -> Perm (Γ :: 𝒢) (Δ :: ℋ)
  | swap : {Γ Δ : Env} {𝒢 : HyperEnv} : Perm (Γ :: Δ :: 𝒢) (Δ :: Γ :: 𝒢)
  | trans {𝒢 ℋ 𝒥 : HyperEnv} : Perm 𝒢 ℋ -> Perm ℋ 𝒥 -> Perm 𝒢 𝒥
```

The key constructor is cons. Unlike ordinary list permutation, it does not require the head environments to be syntactically identical, it only requires them to be permutation-equivalent. Thus `HyperEnv.Perm` is a deep permutation relation, propagating equivalence both across the ordering of component environments and within the environments themselves.

Well-formedness. Hyperenvironment well-formedness is decomposed into two predicates. The predicate `HyperEnv.Nodup` requires each component environment to be internally linear, while `HyperEnv.PairwiseDisjoint` requires distinct component environments to have disjoint domains:

```
def HyperEnv.Nodup (𝒢 : HyperEnv) : Prop :=
  ∀ Γ, Γ ∈ 𝒢 -> Env.Nodup Γ

def HyperEnv.PairwiseDisjoint (𝒢 : HyperEnv) : Prop :=
  List.Pairwise Env.disjoint 𝒢
```

The distinction is important because the two predicates rule out different violations of linearity. For example, assuming x, y, and z are distinct names:

- $[x : A] \mid_h [y : B, z : C]$ satisfies both predicates.
- $[x : A, x : B] \mid_h [y : C]$ violates `Nodup`, since x appears twice within one component environment.
- $[x : A] \mid_h [x : B, y : C]$ violates `PairwiseDisjoint`, since x appears in two distinct components.

Together, these predicates enforce global linearity: every channel appears at most once across the whole hyperenvironment.

Lifted operations. The operations on environments are lifted once more to hyperenvironments by applying them pointwise to each component environment. Thus, type shifting, name substitution, and type substitution on hyperenvironments follow the same pattern as the corresponding operations on environments. The file `HyperEnvironment/Lemmas/Basic.lean` proves that these lifted operations preserve the structural invariants used later in the development, including names, disjointness, pairwise disjointness, and deep permutation. For example, preservation of deep permutation under shifting is proved by induction on the proof of the hyperenvironment permutation relation, using the corresponding environment-level permutation lemmas. Since these definitions are obtained uniformly from the environment-level construction, they are not reproduced in full here. The complete definitions and proofs are included in the accompanying Lean archive.

4.1.5 Typing Judgements. The typing relation is defined in `Judgement/Basic.lean` as the inductive predicate `Typing`:

```
inductive Typing : Nat -> Proc -> HyperEnv -> Prop
```

We write $n \vdash P :: \mathcal{G}$ for a derivation of this predicate. The index n tracks the current type-binder depth, while the remaining indices record the process being typed and its associated typing environment. This intrinsic indexing is later used by the erasure maps in the semantics and metatheory. The full inductive definition, including the typing constructors and the exchange rules, is given in Section D.3.

Type-depth indexing. The type-depth index is relevant whenever the derivation crosses a polymorphic binder. For example, the `forall_` constructor increments the depth from n to $n+1$ and shifts the surrounding environment to keep embedded type indices in scope:

```
| forall_ :
    Typing (n + 1) P [x:B :: Γ⁺] ->
    Typing n (#x⦅◇⦆.P) [x:∀.B :: Γ]
```

The shift Γ^+ abbreviates shifting the type indices in Γ by one starting at depth 0, mirroring the binder-sensitive shifting used by type substitution in Section 4.1.1.

The depth index is also relevant for the MIX rule, which requires both sub-derivations to share the same depth n. When derivations are available at different depths, they can be brought into alignment using `Typing_weakening`. This lemma states that a derivation at depth n can be lifted to depth $n+c$ by shifting the embedded type indices in both the process and the hyperenvironment:

```
lemma Typing_weakening :
  Typing n P 𝒢 -> ∀ d c, Typing (n + c) (P ↑ d, c) (𝒢 ↑ d, c) := by ...
```

This provides the weakening principle needed to compose derivations across polymorphic binders while keeping embedded type indices in scope.

Exchange and explicit side conditions. In some constructors, the formalised rule chooses an ordering different from the paper presentation so that the entry manipulated by the rule occurs at the head of the list representing its environment component. This is an implementation-oriented choice and does not affect the typing discipline. The order-insensitive interpretation of environments and hyperenvironments is recovered by exchange rules in the typing relation, which permit reasoning up to permutation:

```
| exchange_env   : Typing n P (Γ :: 𝒢) -> Γ ~ Δ -> Typing n P (Δ :: 𝒢)
| exchange_hyper : Typing n P 𝒢 -> 𝒢 ~ ℋ -> Typing n P ℋ
```

The exchange rules recover the unordered character of the paper presentation without quotienting environments or hyperenvironments. The complementary issue is partiality: although merging is implemented as total list concatenation, typing rules carry explicit disjointness and freshness premises to ensure that only valid linear compositions are formed. For example, `mix` requires the two hyperenvironments to be disjoint, while prefix rules such as `tensor` and `parr` require freshness premises ensuring that subject channels do not already appear in the continuation environment.

The same design principle reappears later in the operational semantics, where transitions must also respect permutation equivalence of environments and hyperenvironments.

Co-finite quantification. Binding rules use co-finite quantification as described in Section 3.4. For example, the `parr` constructor types an `input` prefix by requiring the opened continuation to be well typed for every fresh channel name outside a finite set L:

```
| parr (hF : x ∉ Γ.names) (L : Finset FPName) :
    (∀ y ∉ L, Typing n (P⦅#y⦆) [y:A :: x:B :: Γ]) ->
    Typing n (x⦅●⦆.P) [x:A ⅋ B :: Γ]
```

Here, the process carries no explicit bound channel name. Instead, the bound occurrence in the continuation is instantiated by opening P with a fresh name y. The premise `hF` enforces that the subject channel x is not already present in the continuation environment. Rules binding multiple names, such as `cut`, follow the same pattern but quantify over two distinct fresh names.

Structural invariants. `Judgement/Names.lean` establishes the central name invariant for the typing relation, named `Typing_f_eq_names`. It states that, for any well-typed process, the free names of the process coincide exactly with the domain of its hyperenvironment.

$$n \vdash P :: \mathcal{G} \implies P.f = \mathcal{G}.names$$

The proof proceeds by induction on the typing derivation. Most cases follow by unfolding `Proc.f` and `HyperEnv.names` and rewriting with the induction hypothesis. In the co-finite binding cases, the proof instantiates fresh names, applies the induction hypothesis to the opened process, and then uses `Proc.f_open_erase` to cancel the introduced names from the free name set and recover equality for the closed process.

This ensures exact correspondence between process free names and typed environment entries: no free process channel is left untyped, and no environment entry is unused.

Building on this, `Judgement/Properties/Linearity.lean` proves that any well-typed hyperenvironment is pairwise disjoint and all component environments have no duplicate names.

$$n \vdash P :: \mathcal{G} \implies \texttt{Nodup}\ \mathcal{G} \wedge \mathcal{G}.\texttt{PairwiseDisjoint}$$

The proof proceeds by induction on the typing derivation. Most non-binding rules are discharged uniformly by unfolding the relevant definitions and applying the induction hypothesis. The co-finite binding cases (`cut`, `tensor`, `parr`, and contraction `c`) require instantiating fresh names and reconstructing both nodupness and disjointness of the component environments from the opened induction hypothesis. The exchange cases use preservation of these properties under permutation.

Together, `Typing_f_eq_names` and `Typing_preserves_linearity` guarantee that typing derivations can only be constructed for well-formed, linear contexts. In particular, a well-typed process cannot contain duplicate free channel occurrences: by the name correspondence, duplicate free names would require duplicate entries in the hyperenvironment, contradicting pairwise disjointness.

A third invariant, `Typing_preserves_lc`, establishes that every well-typed process is locally closed and that every component environment in its hyperenvironment is locally closed at the current type-binder depth. The proof is split into `Typing_preserves_lc_proc` for the process and `Typing_preserves_lc_context` for the hyperenvironment, then packaged together. Both proceed by induction on the typing derivation. The co-finite binding cases use `Proc.lc_of_open_gen` and `Proc.lc_of_open_two` to recover local closure of the closed process from that of the opened one. The `forall_` and `exists_` cases additionally require `Env.lc_shift_inv_0` and `Types.lc_subst_inv_0` to invert the effect of shifting and substitution on the environment and type indices respectively.

Substitution. `Judgement/Subst.lean` establishes that typing is stable under both forms of substitution. `Typing_substNames` states that if $n \vdash P :: \mathcal{G}$ and y does not appear in $\mathcal{G}$ except possibly as x, then substituting y for x throughout yields a well-typed process under the correspondingly substituted hyperenvironment. `Typing_substTypes` gives the analogous result for type substitution, where substituting a type A for De Bruijn index k throughout a derivation yields a well-typed process at depth $n - 1$. Both proofs proceed by induction on the typing derivation and rely on the commutation lemmas from Section 4.1.2.

4.2 Semantics

The dynamic behaviour of the πMLL fragment is formalised in the `Semantics/` namespace. Following the erasure structure described in Section 2.5, the development contains three related transition systems: a typed transition system over typing derivations, and two projected systems describing the induced process and environment behaviour.

- **`TypingStep`$_\mathtt{m}$**: the primary, resource-aware LTS over typing derivations (`JudgementStep/Basic.lean`);
- **`ProcStep`$_\mathtt{m}$**: the process-level LTS obtained by forgetting typing resources (`ProcStep/Basic.lean`);
- **`EnvStep`$_\mathtt{m}$**: the resource-level LTS describing the corresponding evolution of hyperenvironments (`EnvStep/Basic.lean`).

The complete inductive definitions of these three transition systems are given in Listings 12 to 14.

Labels. Transition labels are formalised in `Labels.lean`, directly realising the label algebra introduced in Section 2.5. Although the operational semantics mechanised here target πMLL, the action vocabulary is defined for the full πLL calculus to support future extensions. The free and introduced name functions f and i are realised as `Lbl.f` and `Lbl.i`, computed by structural recursion on the label. The disjointness condition on `par` labels is enforced as the decidable predicate `Lbl.WF`, which appears as an explicit side condition in the parallel synchronisation rules.

The core action type, `Act`, captures observable communication events, including empty communication, channel transmission, type instantiation, and server interaction. Selection and server actions are parameterised by the auxiliary datatype `Mu`. The `Lbl` inductive then extends observable actions with the silent transition τ, explicit session-link labels, and a parallel constructor combining two observable actions directly, reflecting the synchronized action labels used by the parallel rules.

```
inductive Act : Type
  | one       (x : FPName)
  | tensor    (x y : FPName)
  | output    (x : FPName) (A : Types)
  | muBrack   (x : FPName) (μ : Mu)
  | ...

inductive Lbl : Type
  | tau
  | link  (x y : FPName)
  | act   (p : Act)
  | par   (l l' : Act)
```

Each label defines computable sets of free names (`Lbl.f`) and introduced names (`Lbl.i`), which are used in the side conditions of the rules PAR$_1$, SYN and PAR$_2$. The label notation is overloaded through the `HasBracket` and `HasParen` typeclasses. Since labels record concrete observable actions, their subjects are free process names rather than locally nameless channels. The overloading is therefore confined to the label language. Bracketed and parenthesised labels are distinguished by the type of their arguments, allowing empty communication, channel transmission, type communication, and server actions to share a compact notation. The corresponding notation instances are given in Listing 6.

4.2.1 The Typed Transition System. The primary LTS, $\mathtt{TypingStep}_\mathtt{m}$, is a relation between typing derivations. Its constructors mirror the operational rules shown in Figure 2, but make the type-theoretic content of each step explicit: every transition carries a typing derivation as its source and produces a typing derivation as its target. The system is therefore intrinsically typed.

```
inductive TypingStepₘ :
  Typing n P 𝒢 → Lbl → Typing n' P' 𝒢' → Prop
```

As established previously, both environment and hyperenvironment composition are formalised using total list appends. Consequently, the explicit disjointness proofs required by the static `Typing` relation to safely mix environments must propagate directly into the dynamic $\mathtt{TypingStep}_\mathtt{m}$ relation. For instance, the $\mathtt{par}_1$ constructor requires explicit hypotheses (`hD1` and `hD2`) to guarantee that the hyperenvironments remain strictly disjoint both before and after the transition:

```
| par₁
    (hD1 : 𝒢.disjoint ℋ) (hD2 : 𝒢'.disjoint ℋ) (disj : l.i ∩ Q.f = ∅)
    (h : TypingStepₘ 𝒟 l 𝒟') :
    TypingStepₘ
      (Typing.mix hD1 𝒟 ℰ) l (Typing.mix hD2 𝒟' ℰ)
```

Listing 1. The $\mathtt{par}_1$ constructor of the typed transition relation, with explicit pre- and post-transition disjointness premises.

The post-state disjointness proof `hD2` is included explicitly as a proof-engineering convenience. It is recoverable from the pre-state disjointness proof `hD1`, the freshness condition $i(l) \cap Q.f = \emptyset$, the name correspondence invariant `Typing_f_eq_names`, and $\mathtt{TypingStep}_\mathtt{m}\mathtt{\_names\_bound}$, which states that the target hyperenvironment contains no names beyond those already present in the source or introduced by the label. Keeping `hD2` as a premise avoids reconstructing this derived disjointness fact repeatedly in later proofs, at the cost of making the transition relation slightly less minimal. The transition name bounds are stated in Listing 21.

Strictly stepping typing derivations can expose structural artifacts that are usually hidden in the paper presentation. For example, in Figure 3, the intermediate derivation is typed by $\varnothing \mid y{:}\,1$ rather than simply $y{:}\,1$, because the MIX rule preserves the empty component contributed by the terminated left process. In Lean, this corresponds to a `Typing.mix` constructor containing a $\mathtt{Typing.mix}_\emptyset$ sub-derivation. The result is valid, but reducing it to the canonical form requires applying the monoidal identity laws of the hyperenvironment explicitly, rather than by definitional equality. The final $\equiv$ step in the figure, which additionally uses process congruence to identify $\mathbf{0} \mid \mathbf{0}$ with $\mathbf{0}$, is not currently automated because structural congruence has not been mechanised. This limitation is discussed further in Section 6.

A subtler asymmetry concerns the co-finite quantification style used in the `Typing` and $\mathtt{TypingStep}_\mathtt{m}$ relations. In `Typing`, binding rules use the binder-style notation $\forall y \notin L$ as part of the constructor premise. In $\mathtt{TypingStep}_\mathtt{m}$, the corresponding constructors instead carry the co-finite premise as an implicit field `huniq` and record the concrete instantiation point as a separate explicit premise `hy`. This makes the specific fresh name chosen by the transition visible at the constructor level, which simplifies pattern matching in the typability proof at the cost of a slightly more verbose inductive definition.

4.2.2 Projected Transition Systems. The process and environment transition systems are defined to capture the two erasure views of a typed transition. $\mathtt{ProcStep}_\mathtt{m}$ records the process behaviour obtained by forgetting typing resources, while $\mathtt{EnvStep}_\mathtt{m}$ records the corresponding resource evolution. These relations are later connected to $\texttt{TypingStep}_\texttt{m}$ by the simulation theorems in Section 5.1.

Process steps. $\texttt{ProcStep}_\texttt{m}$ models the syntactic evolution of processes independently of typing resources. Unlike the typing relation, it is not indexed by a typing derivation and therefore does not use co-finite premises. Binder-opening rules instead choose concrete fresh names directly and record the required freshness as explicit side conditions. For example, the `tensor` rule opens the continuation with a concrete name y, requiring y to be fresh for both the subject channel and the continuation:

```
| tensor (hF : y ∉ {x} ∪ P.f) :
    ProcStepₘ (#x⟦•⟧.P) (x⟦y⟧) P⦅#y⦆
```

This creates a mismatch with the co-finite formulation of the typing rules: $\texttt{ProcStep}_\texttt{m}$ records the concrete name chosen by an operational step, while the typing rule provides a premise for all names outside a finite exclusion set. The reconciliation of these two views is handled by the typability proof in Section 5.1.

Environment steps. The structural evolution of hyperenvironments, written $\mathcal{G} \xrightarrow{l} \mathcal{H}$, is formalised by $\texttt{EnvStep}_\texttt{m}$. Unlike the typed transition system, $\texttt{EnvStep}_\texttt{m}$ deliberately contains only the resource transformation described by the label. For example, an input action on an endpoint typed by $\bot$ consumes that resource: $\Gamma, x\colon \bot \xrightarrow{x()} \Gamma$. Similarly, an empty output on an endpoint typed by 1 evolves $x\colon 1$ to the empty environment. These rules describe how session types evolve as actions are performed, independently of any particular process term.

The relation does not independently enforce all disjointness and well-formedness conditions. Those properties are supplied by the typing derivation in $\texttt{TypingStep}_\texttt{m}$. This separation keeps the projected resource semantics lightweight: the environment relation describes how resources change, while the typed semantics proves that these changes occur from well-formed typing environments.

The following metatheory shows that these transition systems agree: typed transitions project to process and environment transitions, and process transitions from well-typed states can be lifted back to typed transitions.

5 Metatheory

This section describes the metatheoretic results connecting the typed, process, and environment transition systems defined in Section 4.2. The development is contained in the `SessionFidelity/` namespace and focuses on session fidelity for the mechanised πMLL fragment.

5.1 Session Fidelity

Session fidelity states that well-typed processes evolve according to their session types, and that their associated linear resources evolve consistently with each process transition. The mechanisation establishes the two erasure simulations from typed transitions to process and environment transitions. The converse lifting direction is developed through typability subject reduction and is complete for the multiplicative cases except `res`. Completing this case would yield the final subject-reduction result.

Erasure simulations. As shown by Montesi and Peressotti [2021, Theorem 4.7], erasure induces a strong bisimulation between lts_p and lts_d. The mechanisation currently establishes the projections from typed derivation steps to their process and environment behaviours. The files `ProcStep.lean` and `EnvStep.lean` contain the corresponding theorems: $\texttt{session\_fidelity\_proc}_\texttt{m}$ projects a typed transition to a process transition, while `session_fidelity_env`$_\mathsf{m}$ projects it to an environment transition. The former corresponds to the forward direction of the erasure result of Montesi and Peressotti [2021], while the latter corresponds to their derivation-level session-fidelity property.

```
theorem session_fidelity_procₘ
  {𝒟 : Typing n P 𝒢} {𝒟' : Typing n' P' 𝒢'}
  (hStep : TypingStepₘ 𝒟 l 𝒟') :
  ProcStepₘ (proc 𝒟) l (proc 𝒟')
```

```
theorem session_fidelity_envₘ
  {𝒟 : Typing n P 𝒢} {𝒟' : Typing n' P' 𝒢'}
  (hStep : TypingStepₘ 𝒟 l 𝒟') :
  EnvStepₘ (env 𝒟) l (env 𝒟')
```

These obligations are discharged by instantiating the co-finite typing premises with sufficiently fresh auxiliary names and applying the typing invariants `Typing_f_eq_names` and `Typing_preserves_linearity`, established in Section 4.1.5. The Lean statements of the resulting process- and environment-erasure theorems are given in Listings 15 and 16.

Typability. The target lifting theorem is formulated in `Typability.lean`. It corresponds to the reverse direction of the erasure correspondence: given a process transition from a well-typed process, the transition should lift to the level of typing derivations. Concretely, if $n \vdash P :: \mathcal{G}$ and $P \xrightarrow{l} P'$, then there should exist a hyperenvironment $\mathcal{G}'$ and a typing derivation $n \vdash P' :: \mathcal{G}'$ connected to the source derivation by a typed transition labelled l. The proof is complete for the multiplicative cases except `res`, whose remaining obligation is discussed in Section 6.

This formulation differs from the paper presentation, which formulates erasure extensionally, using set-valued transition functions and equality of successor sets. In the Lean formalisation, transitions are represented as inductive relations, so the same correspondence is decomposed into relational projection and lifting directions. The theorem `session_fidelity_proc`$_\mathsf{m}$ gives the forward projection from typed derivation steps to process steps, while `typability_subject_reduction`$_\mathsf{m}$ gives the converse lifting direction for processes equipped with a typing derivation. This avoids defining successor sets explicitly in Lean and allows the proof to follow the induction principles generated by the transition relations.

```
theorem typability_subject_reductionₘ
  (𝒟 : Typing n P 𝒢) (hPS : ProcStepₘ P l P') :
  ∃ (𝒢' : HyperEnv) (𝒟' : Typing n P' 𝒢'),
    TypingStepₘ 𝒟 l 𝒟'
```

This theorem forms the technical core of the session fidelity argument. Its proof proceeds by induction on the process transition derivation. Each case inverts the source typing derivation to recover the typing rule compatible with the observed process shape, then constructs a target hyperenvironment and typing derivation for the resulting process.

The proof has two main technical obligations. First, it must reconcile the concrete freshness witnesses used by `ProcStep`$_\mathsf{m}$ with the co-finite premises of the typing rules. For example, a process transition fixes a concrete name introduced by a prefix, while the corresponding typing rule provides a derivation only for all names outside some finite set. Auxiliary *all fresh* lemmas bridge this gap by showing that the co-finite premise can be instantiated with the concrete name appearing in the process step. Second, in the communication and restriction cases, the proof must rearrange hyperenvironments to expose the components containing the communicating endpoints. In particular, synchronization under restriction requires opening the restricted process with fresh endpoints, applying the induction hypothesis to the opened body, and reconstructing the target hyperenvironment up to permutation.

The remaining incomplete case is `res`. In this case, the induction hypothesis yields a typed step for the opened body of the restriction. The missing step is to invert this typed transition to recover the post-state components corresponding to the restricted endpoints, then repackage them into the co-finite premise required by the outer `cut` typing rule. This hyperenvironment reconstruction remains the main outstanding proof obligation. The full Lean statement is given in Listing 17.

The remaining incomplete case is `res`. In this case, inversion of the source typing derivation exposes the body of the restriction under two fresh dual endpoints, giving a premise of the form:

$$n \vdash Q(\!(x, y)\!) :: \mathcal{H} \mid_h [x : A :: \Gamma] \mid_h [y : A^{\perp} :: \Delta]$$

Applying the induction hypothesis to the transition of this opened body yields a typed transition for the opened target body. However, its target hyperenvironment is determined existentially by the induction hypothesis. In order to reconstruct a typing derivation for the closed target process, the proof must recover two distinct target components carrying $x : A$ and $y : A^{\perp}$, since these are precisely the components consumed when the outer `cut` constructor is reintroduced.

The established name-preservation results are insufficient for this reconstruction. In particular, `TypingStep`$_{\mathsf{m}}$`_names_preserved` shows that names not mentioned by the transition label remain present in the target hyperenvironment, but it does not record the environment components in which those names occur. This distinction is essential for restriction: membership of x and y in the post-state name set does not imply that the target judgement can be permuted into a form:

$$\mathcal{H}' \mid_h [x : A :: \Gamma'] \mid_h [y : A^{\perp} :: \Delta']$$

Without this stronger structural conclusion, the co-finite premise required by the outer `cut` rule cannot be reconstructed.

The current proof attempt therefore introduces stronger auxiliary invariants for typed transitions. A single-block preservation lemma is intended to transport an environment component containing an inactive distinguished endpoint through a transition. This is required, for example, in a parallel-transition case where one of the two endpoint-bearing components lies in the stepping operand while the other remains in the unchanged operand. Building on this, a paired-endpoint reconstruction lemma is intended to track the two dual endpoint components simultaneously. Its conclusion permits either that the endpoints remain exposed in distinct components, as required for rebuilding the outer restriction, or that they occur in a merged component while traversing an intermediate transition case. Completing these auxiliary lemmas requires further permutation analysis for parallel, synchronisation, internal communication, and nested restriction transitions.

Consequently, the remaining proof obligation is not caused by the locally nameless treatment of restricted names itself. Fresh representatives for the two bound endpoints can already be selected, and the induction hypothesis can already be applied to the opened body. The unresolved difficulty is structural: the proof must recover enough of the post-transition hyperenvironment decomposition to reintroduce the enclosing restriction. This illustrates a central proof-engineering trade-off of the formalisation: representing hyperenvironments as lists modulo permutation gives a flexible account of exchange, but proofs that depend on the placement of particular resources require dedicated component-extraction and reconstruction lemmas. The full Lean statement is given in Listing 17.

Subject reduction. The target subject-reduction statement is formulated in `SubjectReduction.lean`. It states that, if a well-typed process takes a process transition, then the target process is well typed under some evolved hyperenvironment, and that this hyperenvironment evolves under the same label. Once `typability_subject_reduction`$_{\mathsf{m}}$ is completed, the result follows directly by applying the environment-erasure simulation session_fidelity_env$_m$ to the lifted typed transition.

```
theorem subject_reductionₘ
  (𝒟 : Typing n P 𝒢) (hPS : ProcStepₘ P l P') :
  ∃ 𝒢', EnvStepₘ 𝒢 l 𝒢' ∧ Typing n P' 𝒢'
```

The proof is a direct composition of the previous results: typability_subject_reduction$_m$ lifts the process step to a typed derivation step, and session_fidelity_env$_m$ projects that typed step to the corresponding environment transition. This realises the process-level session fidelity result corresponding to Corollary 4.10 of Montesi and Peressotti [2021].

Interpretation. Together, these results establish the structure of the session fidelity argument for the mechanised fragment, with one remaining gap. The erasure simulations are complete, and subject reduction follows once typability is fully established. The typability theorem is proved for all multiplicative cases except res. In this case, the proof must show that environments not involved in the transition are preserved, up to permutation, across restriction steps. The required infrastructure is largely in place, with the remaining work concentrated in the case analysis relating uninvolved parallel environments to the post-step hyperenvironment produced by the restricted transition. Once this case is closed, the full process-level session fidelity property follows for the mechanised πMLL fragment.

The paper ([Montesi and Peressotti 2021]) additionally establishes behavioural properties including bisimilarity, the diamond property, serialisation, non-interference, and readiness for πMLL. These results are not mechanised in the current formalisation; they are discussed in Section 7.

6 Discussion

This section reflects on the design choices made throughout the formalisation, discusses alternatives that were explored, and identifies limitations of the current mechanisation.

6.1 Proof Architecture

The direction of the typability argument determines the appropriate induction principle. To reconstruct a typed transition from a process step, the proof must follow the operational rule that was performed and then recover the compatible source typing rule. Accordingly, the current proof development for typability_subject_reduction_m proceeds by induction on ProcStep$_m$ and applies typing inversion in each case. This choice is particularly important for parallel composition: the process transition identifies which component steps, whereas induction on the typing derivation initially exposes both typed components without determining which one must be transformed. The resulting architecture aligns the inductive cases with the observed operational behaviour and localises the remaining reconstruction work to the synchronisation and restriction cases.

The role of EnvStep$_m$ was also simplified during development. In an earlier formulation, the proof attempted to invert environment transitions in order to recover the hyperenvironment structure needed for typability. As a consequence, the environment relation had to carry additional freshness and structural premises so that inversion exposed the assumptions required by the corresponding hyperenvironment-extraction lemmas. These premises were useful for proof reconstruction, but were not intrinsic to the resource-level meaning of the environment transition itself. In the current formulation, EnvStep$_m$ is kept as a lightweight description of resource evolution, while the required freshness, linearity, and reconstruction information is supplied by TypingStep$_m$ and the typing invariants. The theorem session_fidelity_env_m then projects typed transitions to their corresponding environment transitions.

6.2 Proof-engineering tradeoffs

Several premises in the typing and transition relations are included for proof convenience rather than logical necessity. The most prominent example is the post-state disjointness proof `hD2` in the parallel constructors $\mathtt{par}_1$ and $\mathtt{par}_2$ of $\mathtt{TypingStep}_{\mathtt{m}}$. The former is shown in Listing 1. This premise is derivable from the pre-state disjointness proof `hD1`, the freshness condition requiring the names introduced by the label to be disjoint from the non-stepping component, and the structural property that transitions introduce only names recorded by their labels. Keeping `hD2` explicit avoids reconstructing this derived fact in every inductive case, at the cost of slightly cluttering the inductive definition. This is representative of a broader proof-engineering tradeoff: some redundancy in the definitions can make individual proofs shorter and more robust, even when it makes the relations less minimal.

A second source of proof overhead is the mismatch between concrete freshness witnesses in $\mathtt{ProcStep}_{\mathtt{m}}$ and co-finite in the typing rules. The binding constructors of $\mathtt{ProcStep}_{\mathtt{m}}$ record the particular names introduced by a transition, whereas the corresponding typing rules are stated co-finitely. The simulation proofs must therefore show that the co-finite typing premise can be instantiated with the concrete names chosen by the process transition. This is handled by auxiliary *all fresh* lemmas, such as those for the `tensor` and `parr` prefix binders. Reformulating $\mathtt{ProcStep}_{\mathtt{m}}$ co-finitely would align the process and typing relations more directly and could reduce the need for these freshness-bridging lemmas, although its effect on the erasure simulation proofs remains to be checked.

6.3 Structural congruence

The formalisation does not include a structural congruence relation for processes. In the paper presentation of πLL, structural congruence identifies processes modulo the commutative, associative, and unit laws of parallel composition, together with scope manipulation for restriction. The mechanisation currently recovers the typing-level counterpart of these laws through the monoidal laws for hyperenvironments and the exchange rules of the typing relation. However, it does not prove that process syntax itself is closed under structural congruence. For example, processes such as $P \mid \mathbf{0}$ and P are not automatically interchangeable. The corresponding simplification must instead be reflected manually at the level of hyperenvironments using the monoidal identity lemmas.

This omission matters because the operational semantics is defined over the concrete syntax tree of processes. In particular, synchronisation rules apply to components exposed by the syntactic structure of parallel composition. Since synchronisation labels are binary, the two processes that synchronise must be visible as sibling components in the syntax tree. For example, in a process of the form $P \mid (Q \mid R)$, a synchronisation between P and Q cannot be exposed by commutativity alone and would require associativity to rearrange the process into $(P \mid Q) \mid R$. A full structural congruence development would therefore require proving that typing and transitions are preserved under an independent congruence relation. An alternative would be to build selected symmetry and reassociation principles directly into the transition relation, for example to account for the orientation of restricted endpoints or the nesting of parallel composition. Whether this would simplify the session-fidelity proofs and later behavioural theory, or merely shift the proof burden elsewhere, remains unexplored.

6.4 Current limitations

The static formalisation covers the full πLL calculus, while the operational semantics and meta-theory currently target πMLL. Within this fragment, the only incomplete case of $\mathtt{typability\_subject\_reduction}_{\mathtt{m}}$ is `res`. As discussed in Section 5.1, this case requires reconstructing the two post-transition hyperenvironment components carrying the restricted dual endpoints so that the enclosing `cut` derivation can be rebuilt. The proof attempt has isolated the missing infrastructure to single-block preservation and paired-endpoint reconstruction through typed transitions; the completed `tensor_parr` extraction lemma in Listing 22 illustrates the same form of permutation-sensitive reasoning for an internal communication case.

Several results from the paper metatheory also remain unmechanised, including the behavioural theory of bisimilarity and congruence, the diamond property, serialisation, non-interference, and readiness. These are discussed as directions for future work in the following section.

7 Future Work

The most immediate directions for future work concern completing the remaining session-fidelity proof obligation and extending the dynamic metatheory to the full πLL calculus.

subsectionCompleting Session Fidelity for πMLL Completing session fidelity for the multiplicative fragment requires both closing the remaining proof gap and reducing the proof-engineering overhead that made this case difficult. The following directions concern the missing restriction case, a possible reformulation of process transitions, and automation of the recurring structural obligations.

Completing the remaining proof case. The immediate remaining proof obligation is the `res` case of $\texttt{typability\_subject\_reduction}_{\texttt{m}}$. As established in Section 5.1, completing this case requires recovering the two endpoint-bearing hyperenvironment components after applying the induction hypothesis beneath a restriction. The current proof attempt isolates this requirement into two auxiliary results: a single-block preservation lemma, needed when only one distinguished component lies beneath a stepping parallel operand, and a paired-endpoint reconstruction lemma, needed to recover the two dual components consumed by the enclosing `cut`. The remaining work lies in completing their permutation-sensitive cases for parallel propagation, synchronisation, internal communication, and nested restriction transitions.

Co-finite quantification in ProcStep. The current $\texttt{ProcStep}_{\texttt{m}}$ relation records concrete fresh names in rules that open bound continuations, while the corresponding typing rules use co-finite quantification. This asymmetry introduces proof overhead in the typability argument, where the concrete names chosen by a process transition must be reconciled with the co-finite premises of the typing derivation. Refactoring the relevant process-transition rules, including `tensor`, `parr`, `one_bot`, `tensor_parr`, and `res`, to use a co-finite formulation could align the two relations more directly and reduce the need for auxiliary *all-fresh* lemmas. Such a reformulation would, however, make the erased process semantics less directly concrete, and its effect on the erasure simulation proofs remains to be investigated.

Proof automation. A recurring cost throughout the formalisation is the manual rearrangement of hyperenvironments using permutation, disjointness, name-set, and freshness lemmas. Many of these obligations amount to routine framing arguments: showing that components remain disjoint, that two hyperenvironments agree up to permutation, or that a chosen name is fresh for the relevant context. A dedicated `aesop` rule set, together with a curated collection of simplification lemmas for normalising name-set and disjointness goals, could automate much of this bookkeeping and significantly reduce proof size. This would not replace the main inversion and reconstruction arguments, such as the remaining `res` case, but could make those arguments substantially more manageable.

7.1 Extending to full πLL

Once the multiplicative fragment is complete, extending the formalisation to the full πLL calculus appears to require additional proof engineering rather than a new formal architecture. The static model is already defined for the full calculus, and many typing inversion lemmas for the additive, polymorphic, axiom, and exponential constructs are already available. The main work is therefore expected to lie in extending `typability_subject_reduction`$_{\mathtt{m}}$ and in proving the hyperenvironment extraction lemmas needed for the additional synchronisation cases, rather than in the erasure projections or the unit-like transition axioms. Some additional work would also be needed to extend the operational semantics with the transition constructors corresponding to the full set of rules from Appendix B.2 of Montesi and Peressotti [2021].

The remaining prefix and unit-like transition axioms from Appendix B.2 of Montesi and Peressotti [2021] are expected to follow the same proof pattern as the already mechanised `one` and `bot` cases. This includes the additive selections, server-use actions, polymorphic send and receive actions, disposal and duplication send actions, and the link actions. In each case, the proof should proceed by inverting the source typing derivation, constructing the corresponding typed transition, and reusing the existing local-closure and freshness infrastructure. The polymorphic cases additionally rely on the type-substitution and shifting lemmas developed for the locally nameless representation, while the link and axiom-cut cases require the name-substitution lemmas. The axiom-cut interaction may require special treatment, since it combines restriction with name substitution; one possible approach is to add an explicit axiom-under-restriction transition constructor, analogous to the existing `one_bot` and `tensor_parr` synchronisation constructors.

The remaining synchronisation cases, such as !w, !c, !?, $\exists\forall$, $\oplus_1\&$, and $\oplus_2\&$, follow the same general restriction-synchronisation pattern as the multiplicative communication cases. They require isolating the interacting components, stepping the opened body, and reconstructing the resulting hyperenvironment up to permutation. Once the extraction lemmas for the current `res` case are completed, these cases are expected to follow by analogous reasoning, with the relevant connectives and environments substituted for their multiplicative counterparts.

The duplication and disposal receive cases are more involved than the other server actions, since they generate a larger continuation process and manipulate dependencies and the typing environment. A prior revision of the formalisation, where server dependencies were stated implicitly, included the auxiliary constructions for building these processes and manipulating the corresponding dependencies and typing environments. Since the well-typedness of those constructions had already been established, parts of this earlier infrastructure may be reusable when extending the current dynamic metatheory.

7.2 Structural congruence

A further direction is to mechanise process-level structural congruence and prove that it preserves typing and transitions. This would address the artificial stuck states that arise from the strict binary-tree structure of the process syntax. In particular, commutativity and associativity of parallel composition are needed for the SYN rule to apply to processes that are not immediate siblings in the syntax tree, as discussed in Section 6. A structural congruence relation would internalise the commutative, associative, and unit laws of parallel composition, as well as scope manipulation for restriction, rather than requiring these rearrangements to be handled manually through exchange rules and hyperenvironment permutation lemmas.

Scope manipulation is especially delicate in the locally nameless setting, since moving a process under a restriction binder requires explicit De Bruijn index management. For example, a scope-extrusion principle of the form $(\boldsymbol{\nu}(\bullet,\bullet)\,P) \mid Q \equiv \boldsymbol{\nu}(\bullet,\bullet)\,(P \mid Q')$ would require Q' to be obtained by shifting the bound channel indices of Q by 2, while leaving its free names unchanged, to account for the two additional channel binders introduced by the restriction. This would require both a channel-level shift operation on bound indices and a process-level traversal lifting that operation through the syntax.

The tradeoff is between modularity and local proof convenience. A full structural congruence relation would require preservation theorems showing that typing and transitions are stable under each congruence principle, but these results would then be reusable throughout the metatheory. A rule-oriented approach instead adds symmetric variants of selected rules, or selected symmetry and reassociation principles, directly to the transition relation. This could account for the congruences most relevant to stepping while preserving the syntax-directed structure of the system, which is useful for proof search in Lean. However, these additional constructors would still introduce extra cases in later proofs. Although unordered collections of processes could avoid some commutativity issues, representing process syntax quotient-wise would make syntax-directed inversion and induction substantially harder in Lean, mirroring the difficulties that led environments to be represented as lists rather than finite sets.

7.3 Readiness, productivity and behavioural metatheory

Beyond session fidelity, the operational metatheory of πLL includes results concerning the availability of interactions, the behaviour of infinite executions, and relations between concurrent process behaviours.

Readiness. One of the unmechanised metatheoretic objectives identified in the project description is progress. In the theory of Montesi and Peressotti [2021], this objective corresponds to the readiness theorem. Theorem 5.20 states that, whenever $\vdash P :: \Gamma_1 \mid \cdots \mid \Gamma_n$, every component Γ_i contains a name x such that P can perform an action whose free names include x. Thus, readiness states that each component of the typed interface of a process offers at least one available interaction. Formalising this result would require proving that, for every component of a well-typed process's hyperenvironment, the process can currently perform a labelled transition involving a channel described by that component. The appropriate Lean proof architecture for establishing this correspondence remains to be developed.

Productivity and potential. The paper additionally proves productivity, which concerns the behaviour of infinite executions. Theorem 5.19 states that every infinite execution of a well-typed process contains infinitely many non-silent transitions. Equivalently, a well-typed process cannot execute internally forever while performing only finitely many observable actions. The proof uses a potential-annotated typing system: Lemma 5.18 bounds the length of any finite sequence of τ-transitions by the potential annotation, while Lemma 5.17 connects annotated typing derivations with ordinary typing derivations. Mechanising productivity would therefore require formalising the annotated typing system, its refinement result, and the potential argument used to bound internal computation. The design of this annotated layer and its integration with the present formalisation remain future work.

Further behavioural results. The paper also establishes behavioural and concurrency results that are not mechanised in the present development. These include congruence results for similarity and bisimilarity, the characterisation connecting similarity with typability, and the parallelism properties of the diamond property, serialisation, and non-interference. Similarity and bisimilarity provide relations for comparing process behaviour, while the parallelism properties describe how independent interactions may be reordered or shown not to interfere with one another. Mechanising these results would require introducing the corresponding behavioural relations and extending the transition-system development with the additional metatheoretic arguments from the paper. The organisation and proof-engineering requirements of such an extension remain to be investigated.

8 Conclusion

This thesis set out to mechanise the syntax, semantics, and metatheory of πLL in Lean. The goal was not to extend the source calculus, but to validate and clarify an existing theoretical framework by making its binding structure, freshness assumptions, linear resource accounting, and operational metatheory explicit in a proof assistant.

The formalisation covers the static structure of the full calculus, including session types, processes, environments, hyperenvironments, and typing judgements. Processes are represented using a locally nameless encoding, while type variables are represented using De Bruijn indices. This avoids explicit reasoning about α-equivalence while preserving readable free channel names in the mechanised syntax. The cost of this choice is the need for local-closure predicates, opening and closing operations, and co-finite reasoning principles.

The operational semantics is mechanised for the multiplicative fragment πMLL through three related transition systems: a typed transition system over typing derivations, a process transition system obtained by erasing typing resources, and an environment transition system describing the corresponding evolution of hyperenvironments. This mirrors the erasure structure of the source theory while adapting it to Lean's relational presentation of inductive transition systems.

The main metatheoretic development establishes the structure of the session-fidelity argument for the mechanised fragment. Typed derivation steps project to both process and environment transitions, and process transitions from well-typed sources can be lifted back to typed derivation steps in the completed multiplicative cases. Subject reduction is then obtained by composing this lifting result with the environment projection. The remaining proof obligation is the `res` case of $\texttt{typability\_subject\_reduction}_{\mathtt{m}}$, where the proof must reconstruct the post-transition hyperenvironment after stepping under restriction.

The mechanisation also exposes several proof-engineering tradeoffs. Representing environments and hyperenvironments as lists makes structural induction and inversion feasible, but requires explicit permutation, disjointness, and freshness reasoning. Similarly, the mismatch between concrete freshness witnesses in $\texttt{ProcStep}_{\mathtt{m}}$ and co-finite premises in the typing rules introduces auxiliary proof obligations. Finally, the absence of process-level structural congruence keeps the transition systems syntax-directed, but requires some rearrangements to be handled manually through exchange rules and hyperenvironment permutation lemmas.

Overall, the development provides a substantial mechanised account of the static structure of full πLL and the dynamic metatheory of its multiplicative fragment. It confirms that the central erasure and session-fidelity architecture of the source theory can be represented in Lean, while isolating the remaining proof infrastructure needed to complete the πMLL development and extend the mechanisation to the full calculus.

References

Brian Aydemir, Arthur Charguéraud, Benjamin C. Pierce, Randy Pollack, and Stephanie Weirich. 2008. Engineering Formal Metatheory. In *Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) *(POPL '08)*. ACM, New York, NY, USA, 3–15. doi:10.1145/1328438.1328443

Brian E. Aydemir, Aaron Bohannon, Matthew Fairbairn, J. Nathan Foster, Benjamin C. Pierce, Peter Sewell, Dimitrios Vytiniotis, Geoffrey Washburn, Stephanie Weirich, and Steve Zdancewic. 2005. Mechanized Metatheory for the Masses: The PoplMark Challenge. In *Theorem Proving in Higher Order Logics*, Joe Hurd and Tom Melham (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 50–65.

Luís Caires and Frank Pfenning. 2010. Session Types as Intuitionistic Linear Propositions. In *CONCUR*. 222–236.

Marco Carbone, Sam Lindley, Fabrizio Montesi, Carsten Schürmann, and Philip Wadler. 2016. Coherence Generalises Duality: A Logical Explanation of Multiparty Session Types. In *27th International Conference on Concurrency Theory, CONCUR 2016, August 23-26, 2016, Québec City, Canada (LIPIcs, Vol. 59)*, Josée Desharnais and Radha Jagadeesan (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 33:1–33:15. doi:10.4230/LIPIcs.CONCUR.2016.33

Arthur Charguéraud. 2012. The Locally Nameless Representation. *Journal of Automated Reasoning* 49, 3 (2012), 363–408. doi:10.1007/s10817-011-9225-2

N. G. de Bruijn. 1972. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem. *Indagationes Mathematicae (Proceedings)* 75, 5 (1972), 381–392. doi:10.1016/1385-7258(72)90034-0

Roberto Di Cosmo and Dale Miller. 2023. Linear Logic. In *The Stanford Encyclopedia of Philosophy* (fall 2023 ed.), Edward N. Zalta and Uri Nodelman (Eds.). Metaphysics Research Lab, Stanford University. https://plato.stanford.edu/archives/fall2023/entries/logic-linear/

Jean-Yves Girard. 1987. Linear Logic. *Theor. Comput. Sci.* 50 (1987), 1–102. doi:10.1016/0304-3975(87)90045-4

Robin Milner, Joachim Parrow, and David Walker. 1992. A Calculus of Mobile Processes, I. *Inf. Comput.* 100, 1 (1992), 1–40.

Fabrizio Montesi and Marco Peressotti. 2021. Linear Logic, the π-calculus, and their Metatheory: A Recipe for Proofs as Processes. *CoRR* abs/2106.11818 (2021). arXiv:2106.11818 doi:10.48550/arXiv.2106.11818

Philip Wadler. 2014. Propositions as sessions. *Journal of Functional Programming* 24, 2–3 (2014), 384–418. Also: ICFP, pages 273–286, 2012.

A Code Availability

The complete Lean development corresponding to this thesis is submitted as a separate archive and is available at: https://github.com/MikkelBrixNielsen/LeanPiLL

The submitted archive corresponds to the version of the formalisation described in this thesis. The online repository may contain later updates, while earlier experimental versions are available in the repository history. These earlier versions are not part of the submitted artefact.

The appendix contains selected excerpts of the development. These excerpts are typeset to closely reflect the corresponding Lean code. In particular, some Unicode notation and spacing is rendered typographically rather than shown as directly executable Lean source. The complete directly executable Lean files, including all definitions, auxiliary lemmas, and proof scripts, are available in the accompanying codebase.

Since the complete Lean development is submitted as an accompanying archive, the appendix contains only definitions and proof fragments required to support the presentation in the thesis. Full implementations and complete proof scripts are provided in the archived source development.

B Lean File Structure

The Lean development is organised into three main parts. The `Model/` directory contains the session types, processes, environments, hyperenvironments, and typing judgement. The `Semantics/` directory contains the transition labels and the process-, environment-, and judgement-level transition systems. The `SessionFidelity/` directory contains the main metatheoretic results relating these transition systems.

```
PiLL/
  Model/
    Environments/
    HyperEnvironments/
    Judgement/
    Processes/
    STypes/
    Base.lean
  Semantics/
    EnvStep/
    JudgementStep/
```

```
    ProcStep/
    Labels.lean
  SessionFidelity/
    ProcStep.lean
    EnvStep.lean
    Typability.lean
    SubjectReduction.lean
```

Listing 2. Top-level structure of the Lean development.

C Common Lean Notation

The notation is purely an overloading layer. Each syntactic category supplies its own instance, while the displayed symbols allow the same operation to be written uniformly throughout the formalisation.

Type Classes. The development uses a collection of typeclasses to overload recurring operations across syntactic categories. This allows shifting, substitution, opening, closing, prefix notation, and permutation to be written uniformly for types, processes, environments, hyperenvironments, actions, and labels. The following listing gives the common notation layer used throughout the later appendix excerpts.

```
class HasShift (Subject : Type) where
  shift : Subject -> Nat -> Nat -> Subject

notation:max S:max " ↑ " d:max ", " c:max => HasShift.shift S d c
notation:max S:max " ↑ " c:max => HasShift.shift S 0 c
notation:max S:max "⁺" => HasShift.shift S 0 1

class HasSubst (Subject Replacement Target : Type) where
  subst : Subject -> Replacement -> Target -> Subject

notation:max S "{" R " // " T "}" => HasSubst.subst S R T

class HasOpen (Subject Replacement Target : Type) where
  open_ : Subject -> Replacement -> Target -> Subject

notation:max S:max "⦅" T " | " R "⦆" => HasOpen.open_ S R T
notation:max S:max "⦅" R "⦆" => HasOpen.open_ S R 0

class HasClose (Subject Replacement Target : Type) where
  close_ : Subject -> Replacement -> Target -> Subject

notation:max S:max "⟪" T " | " R "⟫" => HasClose.close_ S R T
notation:max S:max "⟪" R "⟫" => HasClose.close_ S R 0

class HasOpenTwo (Subject Replacement_1 Replacement_2 Target : Type) where
  open_ : Subject -> Replacement_1 -> Replacement_2 -> Target -> Subject

notation:max S:max "⦅" T " | " R1 ", " R2 "⦆" => HasOpenTwo.open_ S R1 R2 T
notation:max S:max "⦅" R1 ", " R2 "⦆" => HasOpenTwo.open_ S R1 R2 0

class HasCloseTwo (Subject Replacement_1 Replacement_2 Target : Type) where
```

```
  close_ : Subject -> Replacement_1 -> Replacement_2 -> Target -> Subject

notation:max S:max "⟪" T " | " R1 ", " R2 "⟫" => HasCloseTwo.close_ S R1 R2 T
notation:max S:max "⟪" R1 ", " R2 "⟫" => HasCloseTwo.close_ S R1 R2 0

class HasBracket (Subject Content Result : Type) where
   brack : Subject -> Content -> Result

class HasParen (Subject Content Result : Type) where
   paren : Subject -> Content -> Result

notation:80 x"⟦⟧" => HasBracket.brack x ()
notation:80 x"⟦"y"⟧" => HasBracket.brack x y
notation:80 x"⦅⦆" => HasParen.paren x ()
notation:80 x"⦅y⦆" => HasParen.paren x y

class HasPerm (α : Type) where
  perm : α -> α -> Prop

infixr:54 " ~ " => HasPerm.perm
```

Listing 3. Common typeclasses and notation used throughout the Lean development.

Type Notation. The notation in the following listing presents the Lean constructors for session types using the symbols adopted throughout the thesis. In particular, it defines notation for the linear-logic connectives, exponentials, quantifiers, duality, and linear implication, allowing later definitions and theorem statements to closely resemble their mathematical presentation.

```
instance : One Types := ⟨Types.one⟩
instance : Bot Types := ⟨Types.bot⟩
infixr:90 " ⊗ " => Types.tensor
infixr:90 " ⊕ " => Types.oplus
infixr:90 " ⅋ " => Types.parr
infixr:90 " & " => Types.amp
prefix:95 "??" => Types.quest
prefix:95 "!!" => Types.bang
prefix:max "∃." => Types.exists_
prefix:max "∀." => Types.forall_
postfix:max "⊥" => Types.dual
infix:90 " ⊸ " => Types.linImpl
```

Listing 4. Notation for session-type constructors. These declarations provide concrete syntax for the linear-logic connectives, exponentials, quantifiers, duality, and linear implication.

Process Notation. The following listing defines the concrete Lean notation used for process terms. The notation mirrors the process syntax introduced in the theoretical presentation, including prefix forms, restriction, branching, server actions, links, parallel composition, and the inactive process. This notation is used throughout the later typing and transition-system listings.

```
notation:80 x "⟦⟧." P => Proc.one x P
notation:80 x "⟦•⟧." P => Proc.tensor x P
```

```
notation:80 x "⟦" A "⟧." P => Proc.output x P A
notation:80 x "⦇⦈." P => Proc.bot x P
notation:80 x "⦇•⦈." P => Proc.parr x P
notation:80 x "⦇⋄⦈." P => Proc.input x P

notation:75 "ν⦇•,•⦈ " P:80 => Proc.cut P
notation:80 x "⟦L⟧." P:80 => Proc.selectL x P
notation:80 x "⟦R⟧." P:80 => Proc.selectR x P
notation:80 x "⟦USE⟧." P:80 => Proc.consume x P
notation:80 x "⟦DUP⟧⦇•⦈." P:80 => Proc.duplicate x P
notation:80 x "⟦DISP⟧." P:80 => Proc.dispose x P
notation:80 "!" x ".{" P:80 "}" => Proc.server x ∅ P
notation:80 "!" x "⟨" zs "⟩.{" P:80 "}" => Proc.server x zs P
notation:80 x ".case{L" " : " P:80 ", " "R" " : " Q :80"}" => Proc.amp x P Q

notation:80 x "↔p" y => Proc.link x y
infixr:70 " |p " => Proc.par
notation "0" => Proc.nil
```

Listing 5. Notation for process constructors. These declarations provide concrete syntax for prefixes, restriction, branching, server actions, links, parallel composition, and the inactive process.

Label notation. The label syntax reuses the bracket and parenthesis notation introduced for process prefixes. Atomic actions instantiate the common `HasBracket` and `HasParen` interfaces, while generic instances lift this notation from `Act` to `Lbl`. Consequently, the same surface notation can denote either a process prefix or the corresponding transition label, depending on its expected type.

```
notation "L"    => Mu.L
notation "R"    => Mu.R
notation "USE"  => Mu.USE
notation "DUP"  => Mu.DUP
notation "DISP" => Mu.DISP

@[simp] instance : HasBracket FPName Unit Act where
  brack x _ := Act.one x
@[simp] instance : HasBracket FPName FPName Act where
  brack x y := Act.tensor x y
@[simp] instance : HasBracket FPName Types Act where
  brack x A := Act.output x A
@[simp] instance : HasBracket FPName Mu Act where
  brack x μ := Act.muBrack x μ

@[simp] instance : HasParen FPName Unit Act where
  paren x _ := Act.bot x
@[simp] instance : HasParen FPName FPName Act where
  paren x y := Act.parr x y
@[simp] instance : HasParen FPName Types Act where
  paren x A := Act.input x A
@[simp] instance : HasParen FPName Mu Act where
  paren x μ := Act.muParen x μ
```

```
@[simp] instance {S C : Type} [HasBracket S C Act] :
    HasBracket S C Lbl where
  brack s c := Lbl.act (HasBracket.brack s c)

@[simp] instance {S C : Type} [HasParen S C Act] :
    HasParen S C Lbl where
  paren s c := Lbl.act (HasParen.paren s c)

notation:80 x " ↔₁ " y => Lbl.link x y
notation:70 l " |₁ " l' => Lbl.par l l'
notation:80 "τ" => Lbl.tau
```

Listing 6. Notation instances for actions and transition labels. Atomic action notation is lifted to labels so that transition labels mirror process-prefix notation.

D Selected Core Definitions

This appendix collects representative Lean definitions for the core syntactic objects used throughout the formalisation. The listings are intended to show the shape of the mechanised syntax and judgements.

D.1 Session Types

This listing gives the core Lean representation of session types. Type variables are represented using the locally nameless approach discussed in Section 3.2, separating free type variables from bound De Bruijn indices.

```
abbrev Atom := Nat
abbrev FTVar := Nat
abbrev BTVar := Nat

inductive TVar : Type
  | free    (x : FTVar)
  | bound   (x : BTVar)
deriving DecidableEq, Repr

inductive Types : Type where
  | atom        (a : Atom)
  | atomDual    (a : Atom)
  | var         (v : TVar)
  | varDual     (v : TVar)
  | one
  | bot
  | tensor      (A B : Types)
  | parr        (A B : Types)
  | oplus       (A B : Types)
  | amp         (A B : Types)
  | bang        (A : Types)
  | quest       (A : Types)
  | forall_     (A : Types)
  | exists_     (A : Types)
```

```
deriving DecidableEq, BEq, Repr
```

Listing 7. Locally nameless representation of type variables and the Lean definition of session types.

D.2 Processes

This listing gives the core Lean representation of channels and process syntax. Channels are represented using the locally nameless approach discussed in Section 3, separating free names from bound indices.

```
abbrev FPName := Nat
abbrev BPName := Nat

inductive Channel : Type where
  | free    (x : FPName)
  | bound   (x : BPName)
deriving DecidableEq, BEq, Repr

prefix:max "#" => Channel.free
prefix:max "$" => Channel.bound

inductive Proc : Type where
  | nil
  | one         (x : Channel) (P : Proc)
  | bot         (x : Channel) (P : Proc)
  | tensor      (x : Channel) (P : Proc)
  | parr        (x : Channel) (P : Proc)
  | cut         (P : Proc)
  | par         (P Q : Proc)
  | selectL     (x : Channel) (P : Proc)
  | selectR     (x : Channel) (P : Proc)
  | amp         (x : Channel) (P Q : Proc)
  | output      (x : Channel) (P : Proc) (A : Types)
  | input       (x : Channel) (P : Proc)
  | server      (x : Channel) (zs : Finset Channel) (P : Proc)
  | consume     (x : Channel) (P : Proc)
  | duplicate   (x : Channel) (P : Proc)
  | dispose     (x : Channel) (P : Proc)
  | link        (x y : Channel)
deriving DecidableEq, BEq
```

Listing 8. Locally nameless representation of channels and the Lean definition of process syntax.

D.3 Judgements

This listing gives the Lean definition of the main typing judgement, including the constructors for the static πLL typing relation and the exchange rules used to account for list-based environments.

```
inductive Typing : Nat -> Proc -> HyperEnv -> Prop where
  | mix₀ {n : Nat} :
      Typing n 0 ∅
```

```
| mix {𝒢 ℋ : HyperEnv} {P Q : Proc} {n : Nat} (hD : 𝒢.disjoint ℋ) :
    Typing n P 𝒢 -> Typing n Q ℋ ->
    Typing n (P |ₚ Q) (𝒢 |ₕ ℋ)

| one {P : Proc} {x : FPName} {n : Nat} :
    Typing n P ∅ ->
    Typing n (#x⟦⟧.P) [[x:1]]

| bot {Γ : Env} {P : Proc} {x : FPName} {n : Nat} (hF : x ∉ Γ.names) :
    Typing n P [Γ] ->
    Typing n (#x⦅⦆.P) [x:⊥::Γ]

| cut {𝒢 : HyperEnv} {Γ Δ : Env} {P : Proc} {A : Types} {n : Nat}
  (L : Finset FPName) :
    (∀ x ∉ L, ∀ y ∉ L, x ≠ y ->
      Typing n (P⦅#x, #y⦆) (𝒢 |ₕ [x:A::Γ] |ₕ [y:A⊥::Δ])) ->
    Typing n (ν⦅•,•⦆ P) (𝒢 |ₕ [Γ,Δ])

| tensor {Γ Δ : Env} {P : Proc} {x : FPName} {A B : Types} {n : Nat}
    (hF : x ∉ Γ.names ∧ x ∉ Δ.names) (L : Finset FPName) :
    (∀ y ∉ L, Typing n (P⦅#y⦆) ([y:A::Γ] |ₕ [x:B::Δ])) ->
    Typing n (#x⟦•⟧.P) [x:A ⊗ B::Γ,Δ]

| parr {Γ : Env} {P : Proc} {x : FPName} {A B : Types}
    {n : Nat} (hF : x ∉ Γ.names) (L : Finset FPName) :
    (∀ y ∉ L, Typing n (P⦅#y⦆) [y:A::x:B::Γ]) ->
    Typing n (#x⦅•⦆.P) [x:A ⅋ B::Γ]

| oplus₁ {Γ : Env} {P : Proc} {x : FPName} {A B : Types} {n : Nat} :
    B.lc n -> Typing n P [x:A::Γ] ->
    Typing n (#x⟦L⟧.P) [x:A ⊕ B::Γ]

| oplus₂ {Γ : Env} {P : Proc} {x : FPName} {A B : Types} {n : Nat} :
    A.lc n -> Typing n P [x:B::Γ] ->
    Typing n (#x⟦R⟧.P) [x:A ⊕ B::Γ]

| amp {Γ : Env} {P Q : Proc} {x : FPName} {A B : Types} {n : Nat} :
    Typing n P [x:A::Γ] -> Typing n Q [x:B::Γ] ->
    Typing n (#x.case{L : P, R : Q}) [x:A & B::Γ]

| quest {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat} :
    Typing n P [x:A::Γ] ->
    Typing n (#x⟦USE⟧.P) [x:??A::Γ]

| bang {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat} :
    ?ₑΓ -> Typing n P [x:A::Γ] ->
    Typing n (!#x⟨Γ.names.image Channel.free⟩.{P}) [x:!!A::Γ]

| w {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat}
    (hF : x ∉ Γ.names) :
    A.lc n -> Typing n P [Γ] ->
    Typing n (#x⟦DISP⟧.P) [x:??A::Γ]
```

```
  | c {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat}
      (hF : x ∉ Γ.names) (L : Finset FPName) :
      (∀ y ∉ L, Typing n P⦅#y⦆ [x:??A::y:??A::Γ]) ->
      Typing n (#x⟦DUP⟧⦅●⦆.P) [x:??A::Γ]

  | exists_ {Γ : Env} {P : Proc} {x : FPName} {A B : Types} {n : Nat} :
      A.lc n -> Typing n P [x:B{A // 0}::Γ] ->
      Typing n (#x⟦A⟧.P) [x:∃.B::Γ]

  | forall_ {Γ : Env} {P : Proc} {x : FPName} {B : Types} {n : Nat} :
      Typing (n + 1) P [x:B::Γ⁺] ->
      Typing n (#x⦅◇⦆.P) [x:∀.B::Γ]

  | ax {x y : FPName} {A : Types} {n : Nat} (hneq : x ≠ y) :
      A.lc n ->
      Typing n (#x ↔ₚ #y) [x:A⊥:: [y:A]]

  | exchange_env {𝒢 : HyperEnv} {Γ Δ : Env} {P : Proc} {n : Nat} :
      Typing n P (Γ::𝒢) -> Γ ~ Δ ->
      Typing n P (Δ::𝒢)

  | exchange_hyper {𝒢 ℋ : HyperEnv} {P : Proc} {n : Nat} :
      Typing n P 𝒢 -> 𝒢 ~ ℋ ->
      Typing n P ℋ

notation:50 n " ⊢ " P " ∷ " 𝒢 => Typing n P 𝒢
```

Listing 9. The main typing judgement. The judgement Typing n P G states that process P is well typed under hyperenvironment G at type-variable depth n.

```
def proc {𝒢 : HyperEnv} {P : Proc} {n : Nat}
  (_ : n ⊢ P ∷ 𝒢) : Proc := P

def env {𝒢 : HyperEnv} {P : Proc} {n : Nat}
  (_ : n ⊢ P ∷ 𝒢) : HyperEnv := 𝒢
```

Listing 10. Projection functions from typing derivations. Since the judgement is indexed by both the process and hyperenvironment, these projections simply recover the corresponding indices.

D.4 Transition Labels

This listing gives the core Lean definitions of the label and action datatypes discussed the paragraph on labels in Section 4.2, together with the associated free-name, introduced-name, and well-formedness functions used by the operational semantics.

```
inductive Mu : Type
  | L
  | R
  | DISP
  | DUP
```

```
  | USE
deriving Repr, DecidableEq

inductive Act : Type
  | one       (x : FPName)
  | bot       (x : FPName)
  | tensor    (x y : FPName)
  | parr      (x y : FPName)
  | output    (x : FPName) (A : Types)
  | input     (x : FPName) (A : Types)
  | muBrack   (x : FPName) (μ : Mu)
  | muParen   (x : FPName) (μ : Mu)
deriving Repr, DecidableEq

inductive Lbl : Type
  | tau
  | link  (x y : FPName)
  | act   (p : Act)
  | par   (l l' : Act)
deriving Repr, DecidableEq

@[simp] def fNamesAct : Act -> Finset FPName -- free names
  | .one x | .bot x
  | .tensor x _ | .parr x _
  | .input x _ | .output x _
  | .muBrack x _  | .muParen x _ => {x}

@[simp] def iNamesAct : Act -> Finset FPName -- introduced names
  | .one _ | .bot _ | .input _ _ | .output _ _
  | Act.muBrack _ _ | Act.muParen _ _ => ∅
  | .tensor _ y | .parr _ y => {y}

@[simp] def Lbl.f : Lbl -> Finset FPName
  | .tau        => ∅
  | .link x y   => {x, y}
  | .act a      => fNamesAct a
  | .par l l'   => fNamesAct l ∪ fNamesAct l'

@[simp] def Lbl.i : Lbl -> Finset FPName
  | .tau        => ∅
  | .link _ _   => ∅
  | .act a      => iNamesAct a
  | .par l l'   => iNamesAct l ∪ iNamesAct l'

@[simp] def Lbl.WF : Lbl -> Prop
  | .tau => True
  | .link _ _ => True
  | .act _ => True
  | .par l l' => (iNamesAct l) ∩ (iNamesAct l') = ∅
```

Listing 11. Definitions of transition labels, actions, and their associated free and introduced names.

D.5 Transition Systems

This listing collects the full inductive definitions of the three transition relations for πMLL, as referenced from Section 4.2.

```
inductive TypingStep_m :
    {n : Nat} -> {𝒢 : HyperEnv} -> {P : Proc} -> Typing n P 𝒢 -> Lbl ->
    {n' : Nat} -> {𝒢' : HyperEnv} -> {P' : Proc} -> Typing n' P' 𝒢' -> Prop where
  | one {P : Proc} {x : FPName} {n : Nat} {𝒟 : Typing n P ∅} :
      TypingStep_m (Typing.one (x := x) 𝒟) (x⟦⟧) 𝒟

  | bot {Γ : Env} {P : Proc} {x : FPName} {n : Nat}
      {hF : x ∉ Γ.names} {𝒟 : Typing n P [Γ]} :
      TypingStep_m (Typing.bot (x := x) hF 𝒟) (x⦇⦈) 𝒟

  | tensor {Γ Δ : Env} {P : Proc} {x y : FPName} {A B : Types} {n : Nat}
      {hF : x ∉ Γ.names ∧ x ∉ Δ.names} {L : Finset FPName}
      {huniq : ∀ z, z ∉ L -> Typing n (P⦇#z⦈) ([z:A::Γ] |_h [x:B::Δ])}
      {hy : y ∉ ({x} ∪ P.f ∪ Γ.names ∪ Δ.names)} :
      TypingStep_m (Typing.tensor hF L huniq) (x⟦y⟧) (Typing_tensor_all_fresh huniq y hy
    )

  | parr {Γ : Env} {P : Proc} {x y : FPName} {A B : Types} {n : Nat}
      {hF : x ∉Γ.names} {L : Finset FPName}
      {huniq : ∀ z, z ∉ L -> Typing n (P⦇#z⦈) [z :A::x :B::Γ]}
      {hy : y ∉({x} ∪ P.f ∪Γ.names)} :
      TypingStep_m (Typing.parr hF L huniq) (x⦇y⦈) (Typing_parr_all_fresh huniq y hy)

  | par_1 {𝒢 ℋ 𝒢′ : HyperEnv} {P Q P' : Proc} {l : Lbl} {n : Nat}
      {hD1 : 𝒢.disjoint ℋ} {hD2 : 𝒢′.disjoint ℋ}
      {𝒟 : Typing n P 𝒢} {𝒟' : Typing n P' 𝒢′} {ℰ : Typing n Q ℋ}
      (h : TypingStep_m 𝒟 l 𝒟') (disj : l.i ∩ Q.f = ∅) :
      TypingStep_m (Typing.mix hD1 𝒟 ℰ) l (Typing.mix hD2 𝒟′ ℰ)

  | par_2 {𝒢 ℋ ℋ′ : HyperEnv} {P Q Q' : Proc} {l : Lbl} {n : Nat}
      {hD1 : 𝒢.disjoint ℋ} {hD2 : 𝒢.disjoint ℋ′}
      {𝒟 : Typing n P 𝒢} {ℰ : Typing n Q ℋ} {ℰ' : Typing n Q' ℋ′}
      (h : TypingStep_m ℰ l ℰ') (disj : l.i ∩ P.f = ∅) :
      TypingStep_m (Typing.mix hD1 𝒟 ℰ) l (Typing.mix hD2 𝒟 ℰ')

  | syn {𝒢 𝒢′ ℋ ℋ′ : HyperEnv} {P P' Q Q' : Proc} {l l' : Act} {n : Nat}
      {hD1 : 𝒢.disjoint ℋ} {hD2 : 𝒢′.disjoint ℋ′}
      {𝒟 : Typing n P 𝒢} {𝒟' : Typing n P' 𝒢′}
      {ℰ : Typing n Q ℋ} {ℰ' : Typing n Q' ℋ′}
      (h_1 : TypingStep_m 𝒟 l 𝒟') (h_2 : TypingStep_m ℰ l' ℰ')
      (disj : (l |_1 l').i ∩ (P |_p Q).f = ∅) (WF : (l |_1 l').WF) :
      TypingStep_m (Typing.mix hD1 𝒟 ℰ) (l |_1 l') (Typing.mix hD2 𝒟′ ℰ')

  | one_bot {𝒢 : HyperEnv} {Γ : Env} {P P' : Proc} {n : Nat} {L : Finset FPName}
      {huniq : ∀ x ∉ L, ∀ y ∉ L, x ≠ y ->
        Typing n (P⦇#x, #y⦈) (𝒢 |_h [x:1::∅] |_h [y:⊥::Γ])}
      {x y : FPName}
      {hx : x ∉ ({y} ∪ P.f ∪ 𝒢.names ∪ Γ.names)}
```

```
    {hy : y ∉ (P.f ∪ 𝒢.names ∪ Γ.names)}
    {𝒟' : Typing n P' (𝒢 ∣ₕ [∅,Γ])}
    (hStep : TypingStepₘ (Typing_one_bot_all_fresh huniq x y hx hy)
      (x⟦⟧ ∣₁ y⦅⦆) 𝒟') :
    TypingStepₘ (Typing.cut L huniq) (τ) 𝒟'

| tensor_parr {𝒢 : HyperEnv} {Γ Δ Ξ : Env} {P P' : Proc} {A B : Types}
    {n : Nat} {L : Finset FPName}
    {huniq : ∀ x ∉ L, ∀ y ∉ L, x ≠ y ->
      Typing n (P⦅#x, #y⦆)
        (𝒢  ∣ₕ [x:A⊗B::Γ,Δ] ∣ₕ [y:A⊥⅋B⊥::Ξ])}
    {huniq' : ∀ x ∉ L, ∀ y ∉ L, x ≠ y ->
              ∀ x' ∉ L, ∀ y' ∉ L, x' ≠ y' ->
              x ≠ x' -> x ≠ y' -> y ≠ x' -> y ≠ y' ->
      Typing n (P'⦅2 | #x, #y⦆⦅#x', #y'⦆)
        (𝒢 ∣ₕ [x:B::Δ] ∣ₕ [x':A::Γ] ∣ₕ [y':A⊥::y:B⊥::Ξ])}
    {x y : FPName} (hx : x ∉ L) (hy : y ∉ L) (hneq : x ≠ y)
    {x' y' : FPName} (hx' : x' ∉ L) (hy' : y' ∉ L) (hneq' : x' ≠ y')
    (hxx' : x ≠ x') (hxy' : x ≠ y') (hyx' : y ≠ x') (hyy' : y ≠ y')
    (hxP : x ∉ P.f) (hyP : y ∉ P.f) (hx'P : x' ∉ P.f) (hy'P : y' ∉ P.f)
    (hxP' : x ∉ P'.f) (hyP' : y ∉ P'.f) (hx'P' : x' ∉ P'.f) (hy'P' : y' ∉ P'.f)
    (hStep : TypingStepₘ (huniq x hx y hy hneq)
      (x⟦x'⟧ ∣₁ y⦅y'⦆)
      (huniq' x hx y hy hneq x' hx' y' hy' hneq' hxx' hxy' hyx' hyy')) :
    TypingStepₘ (Typing.cut L huniq) (τ)
      (Typing_double_cut_tensor_parr huniq')

| res {𝒢 𝒢' : HyperEnv} {Γ Γ' Δ Δ' : Env} {P P' : Proc} {A : Types}
    {n : Nat} {l : Lbl} {L L' : Finset FPName}
    {huniq : ∀ z ∉ L, ∀ w ∉ L, z ≠ w ->
      Typing n (P⦅#z, #w⦆) (𝒢 ∣ₕ [z:A::Γ] ∣ₕ [w:A⊥::Δ])}
    {huniq' : ∀ z ∉ L', ∀ w ∉ L', z ≠ w ->
      Typing n (P'⦅#z, #w⦆) (𝒢' ∣ₕ [z:A::Γ'] ∣ₕ [w:A⊥ :: Δ'])}
    {x y : FPName} (hneq : x ≠ y)
    (hx_pre : x ∉ P.f ∪ 𝒢.names ∪ Γ.names ∪ Δ.names)
    (hy_pre : y ∉ P.f ∪ 𝒢.names ∪ Γ.names ∪ Δ.names)
    (hx_post : x ∉ P'.f ∪ 𝒢'.names ∪ Γ'.names ∪ Δ'.names)
    (hy_post : y ∉ P'.f ∪ 𝒢'.names ∪ Γ'.names ∪ Δ'.names)
    (hlx : x ∉ l.f ∪ l.i) (hly : y ∉ l.f ∪ l.i)
    (hStep : TypingStepₘ
      (Typing_res_all_fresh huniq x y hx_pre hy_pre hneq) l
      (Typing_res_all_fresh huniq' x y hx_post hy_post hneq)) :
    TypingStepₘ (Typing.cut L huniq) l (Typing.cut L' huniq')

| perm_env {𝒢 ℋ : HyperEnv} {Γ Γ' : Env} {P P' : Proc} {n n' : Nat} {l : Lbl}
    {𝒟 : n ⊢ P :: Γ::𝒢} {𝒟' : n ⊢ P' :: ℋ} (hP1 : Γ ~ Γ')
    (hTS : TypingStepₘ 𝒟 l 𝒟') :
    TypingStepₘ (Typing.exchange_env 𝒟 hP1) l 𝒟'

| perm_hyper {𝒢 𝒢' ℋ ℋ' : HyperEnv} {P P' : Proc} {n n' : Nat} {l : Lbl}
    {𝒟 : n ⊢ P ⊢ 𝒢} {𝒟' : n' ⊢ P' ⊢ ℋ}
    (hP1 : 𝒢 ~ 𝒢') (hP2 : ℋ ~ ℋ')
```

```
    (hTS : TypingStep_m 𝒟 l 𝒟') :
    TypingStep_m (Typing.exchange_hyper 𝒟 hP1) l
      (Typing.exchange_hyper 𝒟' hP2)
```

Listing 12. The typed transition relation TypingStep$_m$.

```
inductive EnvStep_m : HyperEnv -> Lbl -> HyperEnv -> Prop where
  | one {x : FPName} :
    EnvStep_m [[x:1]] (x[[]]) ∅

  | bot {Γ : Env} {x : FPName} :
    EnvStep_m [x:⊥::Γ] (x(())) [Γ]

  | tensor {Γ Δ : Env} {x x' : FPName} {A B : Types}
    (hF : x' ∉ HyperEnv.names [x:A⊗B::Γ,Δ]) :
    EnvStep_m [x:A⊗B::Γ,Δ] (x[[x']]) ([x':A::Γ] |_h [x:B::Δ])

  | parr {Γ : Env} {x x' : FPName} {A B : Types}
    (hF : x' ∉ HyperEnv.names [x:A⅋B::Γ]) :
    EnvStep_m [x:A⅋B::Γ] (x((x'))) [x':A::x:B::Γ]

  | par₁ {𝒢 𝒢' ℋ : HyperEnv} {l : Lbl} :
    EnvStep_m 𝒢 l 𝒢' ->
    EnvStep_m (𝒢 |_h ℋ) l (𝒢' |_h ℋ)

  | par₂ {𝒢 ℋ ℋ' : HyperEnv} {l : Lbl} :
    EnvStep_m ℋ l ℋ' ->
    EnvStep_m (𝒢 |_h ℋ) l (𝒢 |_h ℋ')

  | syn {𝒢 𝒢' ℋ ℋ' : HyperEnv} {l l' : Act} :
    EnvStep_m 𝒢 l 𝒢' -> EnvStep_m ℋ l' ℋ' ->
    EnvStep_m (𝒢 |_h ℋ) (l |_l l') (𝒢' |_h ℋ')

  | one_bot {𝒢 : HyperEnv} {Γ : Env} {x y : FPName} :
    EnvStep_m (𝒢 |_h [[x:1]] |_h [y:⊥::Γ]) (x[[]] |_l y(())) (𝒢 |_h [Γ]) ->
    EnvStep_m (𝒢 |_h [Γ]) (τ) (𝒢 |_h [Γ])

  | tensor_parr {𝒢 : HyperEnv} {Γ Δ Ξ : Env}
    {x x' y y' : FPName} {A B : Types} :
    EnvStep_m
      (𝒢 |_h [x:A⊗B::Γ,Δ] |_h [y:A^⊥⅋B^⊥::Ξ])
      (x[[x']] |_l y((y')))
      (𝒢 |_h [x:B::Δ] |_h [x':A::Γ] |_h [y':A^⊥::y:B^⊥::Ξ]) ->
    EnvStep_m (𝒢 |_h [Γ,Δ,Ξ]) (τ) (𝒢 |_h [Γ,Δ,Ξ])

  | res {𝒢 𝒢' : HyperEnv} {Γ Γ' Δ Δ' : Env} {x y : FPName} {A : Types}
    {l : Lbl} (hFx : x ∉ l.i ∪ l.f) (hFy : y ∉ l.i ∪ l.f) :
    EnvStep_m
      (𝒢 |_h [x:A::Γ] |_h [y:A^⊥::Δ])
      l
      (𝒢' |_h [x:A::Γ'] |_h [y:A^⊥::Δ']) ->
    EnvStep_m (𝒢 |_h [Γ,Δ]) l (𝒢' |_h [Γ',Δ'])
```

```
  | perm_env {𝒢 ℋ : HyperEnv} {Γ Γ' : Env} {l : Lbl} (hP : Γ ~ Γ') :
      EnvStep_m (Γ::𝒢) l ℋ ->
      EnvStep_m (Γ'::𝒢) l ℋ

  | perm_hyper {𝒢 𝒢' ℋ ℋ' : HyperEnv} {l : Lbl} (hP1 : 𝒢 ~ 𝒢') (hP2 : ℋ ~ ℋ') :
      EnvStep_m 𝒢 l ℋ ->
      EnvStep_m 𝒢' l ℋ'
```

Listing 13. The environment-level transition relation $\mathsf{EnvStep}_\mathsf{m}$.

```
inductive ProcStep_m : (P : Proc) -> Lbl -> (P' : Proc) -> Prop where
  | one {P : Proc} {x : FPName} :
      ProcStep_m (#x⟦⟧.P) (x⟦⟧) P

  | bot {P : Proc} {x : FPName} :
      ProcStep_m (#x⦅⦆.P) (x⦅⦆) P

  | tensor {P : Proc} {x y : FPName} (hF : y ∉ {x} ∪ P.f) :
      ProcStep_m (#x⟦•⟧.P) (x⟦y⟧) (P⦅#y⦆)

  | parr {P : Proc} {x y : FPName} (hF : y ∉ {x} ∪ P.f) :
      ProcStep_m (#x⦅•⦆.P) (x⦅y⦆) (P⦅#y⦆)

  | par_1 {P P' Q : Proc} {l : Lbl} :
      ProcStep_m P l P' -> l.i ∩ Q.f = ∅ ->
      ProcStep_m (P |_p Q) l (P' |_p Q)

  | par_2 {P Q Q' : Proc} {l : Lbl} :
      ProcStep_m Q l Q' -> l.i ∩ P.f = ∅ ->
      ProcStep_m (P |_p Q) l (P |_p Q')

  | syn {P P' Q Q' : Proc} {l l' : Act} :
      ProcStep_m P l P' -> ProcStep_m Q l' Q' ->
      (l |_l l').i ∩ (P |_p Q).f = ∅ -> (l |_l l').WF ->
      ProcStep_m (P |_p Q) (l |_l l') (P' |_p Q')

  | one_bot {P P' : Proc} {x y : FPName}
      (hx : x ∉ P.f) (hy : y ∉ P.f) (hneq : x ≠ y) :
      ProcStep_m (P⦅#x, #y⦆) (x⟦⟧ |_l y⦅⦆) P' ->
      ProcStep_m (ν⦅•,•⦆ P) (τ) P'

  | tensor_parr {P P' : Proc} {x x' y y' : FPName}
      (hxP : x  ∉ P.f) (hyP : y  ∉ P.f)
      (hx'P : x' ∉ P.f) (hy'P : y' ∉ P.f)
      (hxP' : x  ∉ P'.f) (hyP' : y  ∉ P'.f)
      (hx'P': x' ∉ P'.f) (hy'P' : y' ∉ P'.f)
      (hxx' : x ≠ x') (hxy : x ≠ y)
      (hxy' : x ≠ y') (hyx' : y ≠ x')
      (hyy' : y ≠ y') (hx'y' : x' ≠ y') :
      ProcStep_m (P⦅#x, #y⦆) (x⟦x'⟧ |_l y⦅y'⦆) (P'⦅2 | #x, #y⦆⦅#x', #y'⦆) ->
      ProcStep_m (ν⦅•,•⦆ P) (τ) (ν⦅•,•⦆ (ν⦅•,•⦆ P'))
```

```
| res {P P' : Proc} {l : Lbl} {x y : FPName} (hneq : x ≠ y)
    (hx : x ∉ P.f ∪ P'.f) (hy : y ∉ P.f ∪ P'.f)
    (hlx : x ∉ l.f ∪ l.i) (hly : y ∉ l.f ∪ l.i) :
    ProcStep_m (P⟪#x, #y⟫) l (P'⟪#x, #y⟫) ->
    ProcStep_m (ν⟪•,•⟫ P) l (ν⟪•,•⟫ P')
```

Listing 14. The process-level transition relation $\mathtt{ProcStep}_{\mathtt{m}}$.

E Session Fidelity Theorems

The following listings give the Lean statements of the completed erasure simulations and the target statements for typability subject reduction and subject reduction for πMLL, as discussed in Section 5.1. The erasure functions `proc` and `env` (Listing 10) extract the process and hyperenvironment from a typing derivation, respectively.

The process- and environment-erasure simulations are complete. The proof of `typability_subject_reduct` is complete for all multiplicative cases except `res`. Consequently, the final subject-reduction result remains conditional on completion of that case.

```
theorem session_fidelity_proc_m
  {n n' : Nat} {𝒢 𝒢' : HyperEnv} {P P' : Proc} {l : Lbl}
  {𝒟 : Typing n P 𝒢} {𝒟' : Typing n' P' 𝒢'} (hStep : TypingStep_m 𝒟 l 𝒟') :
  ProcStep_m (proc 𝒟) l (proc 𝒟') := by ...
```

Listing 15. Process erasure simulation: typed derivation steps project to process steps.

```
theorem session_fidelity_env_m
  {n n' : Nat} {𝒢 𝒢' : HyperEnv} {P P' : Proc} {l : Lbl}
  {𝒟 : Typing n P 𝒢} {𝒟' : Typing n' P' 𝒢'} (hStep : TypingStep_m 𝒟 l 𝒟') :
  EnvStep_m (env 𝒟) l (env 𝒟') := by ...
```

Listing 16. Environment erasure simulation: typed derivation steps project to environment steps.

```
theorem typability_subject_reduction_m
  {n : Nat} {𝒢 : HyperEnv} {P P' : Proc} {l : Lbl}
  (𝒟 : Typing n P 𝒢) (hPS : ProcStep_m P l P') :
  ∃ (𝒢' : HyperEnv) (𝒟' : Typing n P' 𝒢'),
    TypingStep_m 𝒟 l 𝒟' := by ...
```

Listing 17. Target statement for typability subject reduction. The remaining incomplete proof case is `res`

```
theorem subject_reduction_m {n : Nat} {𝒢 : HyperEnv} {P P' : Proc} {l : Lbl}
  (𝒟 : Typing n P 𝒢) (hPS : ProcStep_m P l P') :
  ∃ 𝒢', EnvStep_m 𝒢 l 𝒢' ∧ (Typing n P' 𝒢') := by
  obtain ⟨𝒢', 𝒟', hTS⟩ := typability_subject_reduction_m 𝒟 hPS
  have hES := session_fidelity_env_m hTS
  exact ⟨𝒢', hES, 𝒟'⟩
```

Listing 18. Target statement for subject reduction for the multiplicative fragment, conditional on completion of typability subject reduction.

F Selected Proof Infrastructure

This appendix collects representative lemma statements from the proof infrastructure supporting the session fidelity development, as discussed in Section 5.1 and Section 6.

Freshness bridge lemmas. The following lemmas bridge the mismatch between the co-finite quantification used in the typing rules and the concrete fresh names chosen by $\mathtt{ProcStep}_{\mathtt{m}}$. They show that a co-finite typing premise can be instantiated at any concrete name that is fresh for the process and the surrounding environments. These lemmas are used by the typed transition rules for prefix actions, where the process transition records a particular name, while the typing rule provides a premise for all names outside a finite exclusion set.

```
lemma Typing_tensor_all_fresh {n : Nat} {P : Proc} {Γ Δ : Env}
  {x : FPName} {A B : Types} {L : Finset FPName}
  (hT : ∀ z ∉ L, Typing n (P⟪#z⟫) ([z:A::Γ] |_h [x:B::Δ])) :
  ∀ y, y ∉ ({x} ∪ P.f ∪ Γ.names ∪ Δ.names) ->
  Typing n (P⟪#y⟫) ([y:A::Γ] |_h [x:B::Δ]) := by ...

lemma Typing_parr_all_fresh {n : Nat} {P : Proc} {Γ : Env}
  {x : FPName} {A B : Types} {L : Finset FPName}
  (hT : ∀ z ∉ L, Typing n (P⟪#z⟫) ([z:A::x:B::Γ])) :
  ∀ y, y ∉ ({x} ∪ P.f ∪ Γ.names) ->
  Typing n (P⟪#y⟫) ([y:A::x:B::Γ]) := by ...
```

Listing 19. Freshness bridge lemmas for instantiating the co-finite premises of the `tensor` and `parr` typing rules.

All the *all-fresh* lemmas follow the same proof structure. They first choose the name or names required by the relevant constructor fresh from the co-finite exclusion set, the concrete target names, the free names of the process, and the names already present in the typing environment. The co-finite typing premise is then instantiated with these fresh names, and the resulting derivation is renamed to use the desired concrete names by applying substitution preservation for typing. The remaining proof obligations show that the substitutions have no effect on the surrounding typing environment or on the process body, except at the occurrences introduced by opening the process body. These obligations are discharged using the freshness assumptions together with the substitution and opening lemmas for processes and environments.

Typing inversion. Inversion lemmas recover the typing rule used to derive a given judgement. They are used throughout the typability proof to identify the typing rule compatible with the observed process shape.

The proofs proceed by induction on the typing derivation after first generalising the concrete process shape. All constructors whose conclusion cannot have the required process form are discharged by contradiction. The matching constructor case then uses injectivity of the process and channel constructors to recover the equalities between the displayed process and the process appearing in the typing derivation. For `Typing_inv_one`, this directly yields the empty continuation typing premise and a permutation showing that the hyperenvironment contains exactly the singleton environment for `x`. For `Typing_inv_tensor`, the matching case additionally exposes the hidden types, component environments, and co-finite premise of the original `tensor` rule. The exchange cases propagate the inversion result through environment and hyperenvironment permutation, composing the recovered permutation with the exchange permutation introduced by the typing derivation.

```
lemma Typing_inv_one {n : Nat} {P : Proc} {x : FPName} {𝒢 : HyperEnv}
```

```
  (hT : Typing n (#x⟦⟧.P) 𝒢) :
  (𝒢 ~ [[x:1]]) ∧ Typing n P ∅ := by ...

lemma Typing_inv_tensor {n : Nat} {P : Proc} {𝒢 : HyperEnv} {x : FPName}
  (hT : Typing n (#x⟦•⟧.P) 𝒢) :
  ∃ (A B : Types) (Γ Δ : Env) (L : Finset FPName),
    (𝒢 ~ [x:A⊗B::Γ,Δ]) ∧ (∀ z ∉ L, Typing n (P⦅#z⦆) ([z:A::Γ] |_h [x:B::Δ])) := by ...
```

Listing 20. Representative inversion lemmas for the typing relation. The one case recovers a singleton hyperenvironment and continuation typing derivation, while the tensor case recovers the hidden types, environments, and co-finite premise.

Hyperenvironment extraction. Extraction lemmas isolate specific components from a hyperenvironment permutation, exposing the typing environment required for a particular transition case. The following statement is representative of the extraction lemmas used when reconstructing a `tensor\_parr` synchronisation beneath an enclosing restriction in the proof of `typability\_subject\_reduction`$_m$. Since hyperenvironments are represented as lists modulo permutation, the extracted components can appear in different relative positions. The lemma therefore returns a disjunction covering the possible arrangements.

The proof proceeds by extracting component membership from the pre-transition permutation, followed by case analysis on the relative positions of the restricted endpoints. Freshness and inequality assumptions rule out impossible coincidences, while the valid branches reconstruct the resulting hyperenvironment permutations using permutation rotation, merge, and cancellation lemmas.

Typed-transition name bounds. The following lemmas capture the name-level effect of typed transitions. The first states that a transition can introduce no names beyond those explicitly recorded as introduced by its label. The second states that names not involved in the action are preserved in the target hyperenvironment.

```
lemma TypingStep_m_names_bound
    (hStep : TypingStep_m 𝒟 l 𝒟') :
    𝒢'.names ⊆ 𝒢.names ∪ l.i := by ...

lemma TypingStep_m_names_preserved
    (hStep : TypingStep_m 𝒟 l 𝒟') :
    𝒢.names \ (l.f ∪ l.i) ⊆ 𝒢'.names := by ...
```

Listing 21. Name-evolution bounds for typed transitions. A transition introduces no names beyond those recorded by its label, and preserves all source names not involved in the action.

Both proofs proceed by induction on the typed transition derivation. The prefix axioms reduce to finite-set calculations on the names consumed or introduced by their respective labels. The parallel constructors lift the induction hypothesis through hyperenvironment merge, using monotonicity of union and subset. The permutation cases use the fact that permutation preserves the name set of a hyperenvironment. In the restriction case, the induction hypothesis is applied to the transition of the opened body, giving a name bound in terms of the opened hyperenvironments. The freshness assumptions on the restricted endpoints, stating that neither appears in the label, then allow the temporary names introduced by opening to be cancelled from both sides, recovering the bound on the outer closed hyperenvironments.

```
lemma HyperEnv.Perm.extract_tensor_parr_res
  {𝒢 ℋ 𝒢_r : HyperEnv} {Γ Γ' Γ'' Δ Δ' Δ'' Ξ : Env}
  {u v x x' y y' : FPName} {A B C D E : Types}
  (h_pre : 𝒢 |_h [u:E::Γ'] |_h [v:E⊥::Δ'] ~
    𝒢_r |_h [x:A⊗B::Γ,Δ] |_h [y:C⅋D⊥::Ξ])
  (h_post : ℋ |_h [u:E::Γ''] |_h [v:E⊥::Δ''] ~
     𝒢_r |_h [x:B::Δ] |_h [x':A::Γ] |_h [y':C::y:D::Ξ])
  (hux : u ≠ x) (hux' : u ≠ x') (huy : u ≠ y) (huy' : u ≠ y')
  (hvx : v ≠ x) (hvx' : v ≠ x') (hvy : v ≠ y) (hvy' : v ≠ y')
  (huv : u ≠ v) (hFu : u ∉ 𝒢.names) (hFv : v ∉ 𝒢.names)
  (hFu' : u ∉ ℋ.names) (hFv' : v ∉ ℋ.names)
  (huΔ' : u ∉ Δ'.names) (hvΓ' : v ∉ Γ'.names) :
  (∃ 𝒢_n Γ_n Δ_n Ξ_n,
    𝒢 |_h [Γ', Δ'] ~ 𝒢_n |_h [x:A⊗B::Γ_n,Δ_n] |_h [y:C⅋D⊥::Ξ_n] ∧
    ℋ |_h [Γ'', Δ''] ~ 𝒢_n |_h [x:B::Δ_n] |_h [x':A::Γ_n] |_h [y':C::y:D::Ξ_n])
  ∨
  (∃ 𝒢_n Γ_n Δ_n Ξ_n,
    𝒢 |_h [Γ', Δ'] ~ 𝒢_n |_h [x:A⊗B::Γ_n,Δ_n ++ y:C⅋D⊥::Ξ_n] ∧
      (ℋ |_h [Γ'', Δ''] ~ 𝒢_n |_h [x:B::Δ_n] |_h [x':A::Γ_n ++ y':C::y:D::Ξ_n] ∨
       ℋ |_h [Γ'', Δ''] ~ 𝒢_n |_h [x':A::Γ_n] |_h [x:B::Δ_n ++ y':C::y:D::Ξ_n])) := by ...
```

Listing 22. Representative hyperenvironment extraction lemma for the tensor-parr synchronisation case.